FLAD: Byzantine-Robust Federated Learning Based on Gradient Feature Anomaly Detection

Peng Tang, Xiaoyu Zhu, Weidong Qiu, *Member, IEEE*, Zheng Huang, Zhenyu Mu, and Shujun Li, *Senior Member, IEEE*

***Abstract*—Federated Learning (FL) has gained significant attention due to its ability to jointly train global models by exchanging local gradients instead of raw local datasets. However, poisoning attacks have emerged as a severe threat to FL security, where malicious clients submit crafted gradients to compromise the integrity and availability of the model. Although researchers have worked on countering these attacks to achieve Byzantine-robust FL, it remains challenging to balance high accuracy, robustness, and efficiency simultaneously. We propose FLAD, a novel Byzantine-robust FL approach based on gradient feature anomaly detection, which is the first approach that uses neural networks to adaptively learn gradient features and measure feature similarity to counteract various types of poisoning attacks. Specifically, FLAD employs a small clean dataset to bootstrap trust and trains Feature Extraction Models (FEM). With FEM and DBSCAN clustering, abnormal gradients from malicious clients are detected and eliminated. Extensive experiments on both Non-IID and IID datasets demonstrate that FLAD achieves superior accuracy, robustness, efficiency, and generalizability compared to state-of-the-art approaches. Additionally, we implement privacy-preserving FLAD (PFLAD) using CKKS and Random Permutation techniques to ensure transmitted gradient privacy.**

***Index Terms*—Federated learning, poisoning attack, Byzantine-robust, anomaly detection.**

I. Introduction

FEDERATED Learning (FL) has rapidly developed as a solution to the data island problem and gained widespread application. FL offers a distributed framework for data processing and machine learning [1]. Training data is distributed across various client devices (e.g., desktops, mobile phones, IoT devices) that aim to collectively learn a model without sharing their local data. Specifically, each client individually trains and updates a local model using its private data, then transmits pertinent local model variables to the server. The server aggregates these variables to maintain a global model and broadcasts the aggregated results to distributed clients. This process is repeated until model convergence [2], enabling collaborative learning across diverse client datasets.

Received 18 November 2023; revised 17 December 2024; accepted 9 February 2025. Date of publication 17 February 2025; date of current version 11 July 2025. This work was supported by the Science and Technology Innovation Plan Of Shanghai Science and Technology Commission under Grant 24BC3200600. *(Corresponding authors: Xiaoyu Zhu; Weidong Qiu.)*

Peng Tang, Xiaoyu Zhu, Weidong Qiu, Zheng Huang, and Zhenyu Mu are with the Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: zhux1aoyu@sjtu.edu.cn; qiuwd@sjtu.edu.cn).

Shujun Li is with the School of Computing & Institute of Cyber Security for Society (iCSS), University of Kent, CT2 7NZ Canterbury, U.K.

Digital Object Identifier 10.1109/TDSC.2025.3542437

Current researches on FL focus on protocol design [3], and the robustness of FL has not been fully studied. Although FL protects client's private data from direct transmission and shows great development potential, its distributed characteristics and protocol vulnerability also make it vulnerable to various attacks. Model poisoning attack is the common attacks [3]. For model poisoning attack, any malicious client can mislead global model learning process by sending carefully constructed malicious gradients to server, resulting in model classification errors. A poisoned gradient may even control the entire training process of the model, ultimately invalidating the joint learning model [4]. Therefore, a trustworthy FL framework must can effectively prevent model poisoning attacks on global model. Our work will focus on the global model security to propose a Byzantine-robust FL defence against poisoning attacks.

Existing mainstream Byzantine-robust methods are defended by statistical analysis of the data, i.e., the server discriminates malicious gradients based on the statistical similarity of local gradients [5]. However, due to the lack of proper trust bootstrapping in these approaches, the server is prone to identify malicious gradients purposely constructed based on aggregation rules as honest gradients thus misleading the global model learning. FLTrust addresses this deficiency by bootstrapping trust by building server models using trusted datasets on the server [6]. However, FLTrust still suffers from a single way of extracting trust, general and limited features extracted, and modified malicious gradients are still involved in aggregation leading to slow convergence in model training. Moreover, there are other Byzantine-robust FL schemes that center on clipping the magnitude of gradients and adding noise, which also has implications for the accuracy of honest gradients [7], [8].

Our work: We propose FLAD, a Byzantine-robust FL against model poisoning attacks based on anomaly detection. Following the FLTrust's idea [6], FLAD assumes that server has a small clean dataset (called server dataset) to bootstrap trust. FLAD uses neural network-based feature extraction models (FEM) to build anomaly detectors and clustering models with the clean server dataset's features as criteria to identify malicious gradients. Honest gradients are recognized since their features (extracted by FEM) closely align with server dataset criteria. Conversely, distant features indicate poisoned gradients from malicious clients. Specifically, the server uses FEM trained on server dataset for feature extraction, yielding reduced-dimensional vectors for each client's gradients. Effective differentiation of malicious gradients is achieved by

1545-5971 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

clustering algorithm using Euclidean distance, cosine similarity and relative length difference compared to standard features. Furthermore, we combine CKKS homomorphism [9] and Random Permutation to ensure data privacy during FL transmission, called PFLAD.

Our main contributions are summarized below.

- We proposed FLAD, a Byzantine-robust FL based on anomaly detection. FLAD is the first defense approach that uses neural networks to adaptively learn gradient features and measure gradient similarity to counteract various types of poisoning attacks. We have open-sourced FLAD[1].
- FLAD rejects malicious gradients, truly achieving aggregation only for honest gradients. Therefore, attackers' number and the type of attacks can change dynamically during training. FLAD is fully applicable in scenarios with a substantial number and even can detect over half of attackers for poisoning attacks.
- FLAD is comprehensively assessed across diverse Non-IID and IID datasets, various attacks and defense methods, etc. Experimental results reveal FLAD's high global accuracy and strong robustness against a range of poisoning attacks, including adaptive attack.
- Furthermore, we implement privacy-preserving FLAD (PFLAD) model attack defence scheme using CKKS and Random Permutations to ensure transmitted gradient privacy. Therefore, PFLAD achieves both Byzantine-robust and privacy-preserving, making it suitable for environments with unreliable servers and clients.

II. Related Work

A. Poisoning Attacks to FL

Poisoning attacks generally refer to attacking the training phase of global model and misleading its learning process, thus invalidating the final learned model [3], which are divided into targeted and untargeted attack.

Untargeted Attack: Untargeted poisoning attack's objective is to arbitrarily compromise the integrity of target model [3]. *Byzantine attack* is an untargeted poisoning attack that uploads arbitrary carefully constructed malicious gradients to server, which leads to global model training failure. Common untargeted attacks also encompass *Gaussian Attack* [10], *Zero-gradient Attack* [11], *Sign-flipping Attack* [12] and *MPAF Attack* [13]. Moreover, *Adaptive Local Model Untargeted Poisoning Attack* [14] [15] have been proposed, which treats the attack as an optimization problem, and can cause a higher error rate to global model.

Targeted Attack: Targeted attack induction model outputs attacker-specified target labels for specific test samples with triggers, while the model for other clean samples is unaffected. Targeted attacks generally can be achieved by poisoning local datasets and extending malicious local gradients to implant distributed or centralized backdoor in global model. Common targeted poisoning attacks encompass *Label-flipping Attack* [16], [17], *Model-replacement Attack* [18], [19] and *Distributed Backdoor Attack* (DBA) [20]. Label-flipping attack is the process of flipping the labels of normal examples to target labels, but the original features of the data remain unchanged. Backdoor poisoning attack requires the attacker to modify individual features or regions of the original dataset in order to implant backdoor triggers in the data so that the final learning model on clean data can function properly, but will constantly predict the attacker's chosen target label when data containing backdoor triggers are present.

B. Byzantine-Robust FL

Currently, state-of-the-art defenses against FL poisoning attacks can be classified into three main categories.

Robust Aggregation Rules: Traditional FL adopts FedAvg [21] aggregation, where the server broadcasts weighted averages of local gradients as global model to clients. However, FedAvg is highly susceptible to malicious gradients [14]. Therefore, various Byzantine-robust rules have been proposed, including *Krum* [22], *Trimmed-mean*, *Median* [23], *Bulyan* [24], *DETOX* [25] and *RFA* [26]. While most of them are vulnerable to poisoning attacks in specific scenarios, they incur substantial communication overhead [14], [27]. Moreover, rules such as Krum and Bulyan have explicit restrictions on the proportional threshold of malicious attackers.

Correction of Local/Global Gradients: The server's assessment of honesty and malice relies solely on received local gradients during the FL training. Therefore, numerous methods focus on either improving local gradients sent to the server or preprocessing local gradients and clipping global gradients at the server, such as Byrd-SAGA [11], signSGD [28], FLTrust [6], CRFL [7] and Attack-adaptive Aggregation [29]. Byrd-SAGA adjusts local stochastic gradients via averaging before transmission to the server, diminishing noise in honest gradients. In signSGD, clients only need to transmit the sign of their local gradient vectors to the server, and the global model update is determined by majority voting. FLTrust requires server to preprocess the uploaded local gradients by normalization, and then aggregates them according to the weighted average of the trust scores. CRFL achieves the effectiveness and robustness of the global model by clipping and stochastic gradient smoothing of the aggregated global gradients. Attack-adaptive Aggregation dynamically adjusts aggregation weights by training neural networks with attention mechanisms to identify and mitigate potential attacks on federated learning models.

Malicious Node Detection and Rejection: Server can avert contamination of global model by rejecting malicious gradients before aggregation. The research [14] has generalized RONI [30] and TRIM [31] to propose ERR (Error Rate based Rejection) and LFR (Loss Function based Rejection) from the actual effect of the global model, which select the local updates that optimize global model performance to aggregate. However, both schemes are not effective in some scenarios [14]. TRIM [31] is a poisoning attack defense for regression learning, which uses a trimmed loss function to determine training data points with minimal loss relative to the model. Although such defenses based on the actual effect of model can bring high accuracy, the high computational overhead incurred during trials cannot be ignored. Moreover, FLDetector [32] is a method designed to detect malicious clients.

[1] [Online]. Available: https://github.com/tp-sh/FLAD

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

It primarily leverages the Cauchy Mean Value Theorem and the L-BFGS algorithm to analyze the consistency of clients' model updates, thereby identifying potential malicious clients. AUROR [33] uses the K-means to remove local parameters that exceed the threshold based on the difference between the honest and malicious parameter distributions. Relative Weight Distance (RWD) [34] uses relative distance between local gradients and threshold to detect malicious gradients. FLAME [8] uses cosine similarity-based HDBSCAN to initially filter out malicious updates. DeepSight [35] introduces various metrics to measure similarity between local updates, thus performing poisoning filtering on local models by classification and clustering. However, in such threshold-based defences, the detection of malicious client is highly dependent on the setting of threshold.

III. System and Problem Setting

A. Threat Model

Referring to the related work [6], malicious attackers can fully control poisoned clients, i.e., attackers can control the local dataset and the process of local training. These poisoned clients are not required to follow FL protocols and can collude with each other. Furthermore, honest clients and server perform normal operations required by FL protocols and attackers cannot control any normal operation of them. Adaptive local poisoning attack is stronger in the full-knowledge setting [14]. Therefore, we consider this setting to evaluate FLAD in Section V.

B. Defense Goals

A reliable Byzantine-robust FL defense must can effectively prevent malicious clients from poisoning global model. Generally, the FL defense should contain four defensive goals:

- *Accuracy:* In non-adversarial settings, the FL defence should achieve a global model's accuracy equivalent to that learned by FedAvg.
- *Robustness:* In the adversarial settings, defence should ensure that the global model is not misled by attackers and global model's accuracy can be similar to FedAvg without attacks, rendering the poisoning attack invalid.
- *Efficiency:* In contrast to FedAvg under non-adversarial settings, defence must avoid increasing client workload and communication overhead. The client-server interactions should not significantly increased.
- *Generalizability:* Defence needs to be applicable to federated systems with different scenarios, such as different scales of client numbers, communication rounds, and varying poisoning ratios, among others.

C. Defender's Knowledge and Capability

Defense must operates on the server, which lacks access to clients' private datasets, local training processes, and information about the number of malicious clients. However, the server has access to the global model at each interaction. Moreover, following the idea of FLTrust [6], FLAD assumes that server has a small clean dataset D_s (called *server dataset*) to extract honest gradients' features, thus bootstrapping local gradients' trust. Experiments indicate that FLAD maintains high model accuracy even if D_s comprises only 0.5% of training data or has a distribution bias of 0.8, as detailed in Section V. Therefore, server can obtain a less demanding D_s with minimal effort, such as manual generation and labeling [6].

D. System Setting

We assume that clients' number is n and malicious attackers' number is c. The range of honest clients is $1 \sim (n-c)$, while malicious clients is $(n-c+1) \sim n$. To ensure model usability, honest clients' number should be greater than two, which is easily achieved in FL. In the tth iteration, the local gradients received by server is $\{w_1^t; w_2^t; \cdots; w_n^t\}$ and the aggregated model is W^t.

IV. Our FLAD Method

A. Overview of FLAD

Given the significant distinctions between gradients from malicious and honest clients [5], server can construct anomaly detectors using server dataset to extract features from local gradients and categorizes them as honest or malicious. Anomaly detectors comprise two components. The first component is feature extraction, wherein neural networks are trained on server dataset. These networks are used to extract features from local gradients, generating feature vectors with the server dataset as honest standard. The second component is malicious client detection. The server evaluates similarity between feature vectors by DBSCAN [36], enabling effective differentiation and rejection of malicious gradients before aggregation. Fig. 1 shows the framework of FLAD.

B. Anomaly Detector

1) Feature Extraction: Before malicious detection on local gradients, the server trains feature extraction models (FEM) with the server dataset D_s and uses this model to extract local gradients features. The server uses D_s to bootstrap trust instead of relying entirely on client's gradients.

Feature Extraction Model (FEM): We choose fully connected neural network (FCNN) to bulid FEM. FCNN structure is simple, capable of uncovering patterns within seemingly disordered data like gradient weights. FEM's objective is to extract features from honest gradients, enabling it to correctly distinguish between malicious and honest gradients. Furthermore, FEM's training set from D_s is relatively small, so the training time and computational consumption are acceptable for a powerful server. *FEM serves solely as a feature extraction tool for client gradients and does not function as a binary classification model for these gradients. Therefore, FEM's training set exclusively comprises clean server gradients, excluding any malicious ones.*

In training FEM, every class model gradient from D_s is expanded into a one-dimensional vector before being fed into the network. The FEM output is a feature value, indicative of the respective input gradient. If uploaded gradient is expanded with dimension d, FEM's hidden layer dimensions are $\lfloor \frac{d}{10} \rfloor$, $\lfloor \frac{d}{100} \rfloor$,....., 1, with input layer dimension d.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

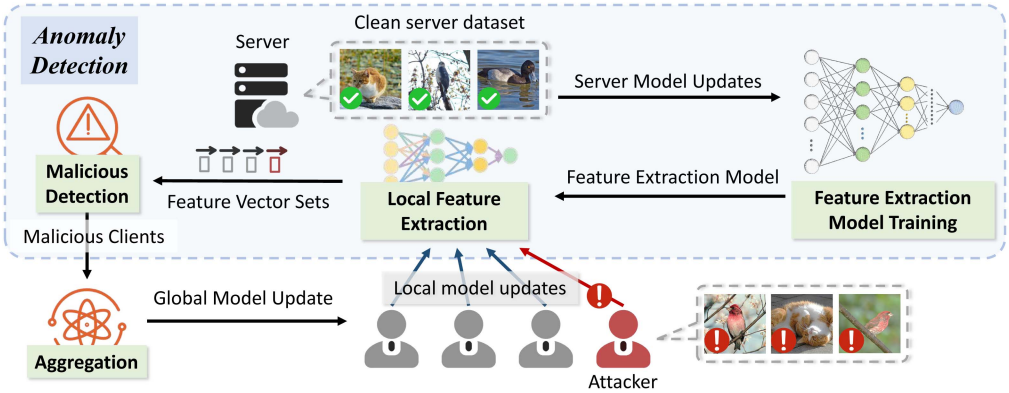


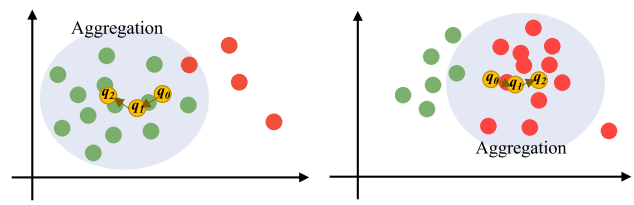
Fig. 1. The framework of FLAD.

Algorithm 1: Feature Extraction Model Training.

Input: W^{t-1}: Global model gradients at $t-1$th round;
D_s: Server dataset for bootstrapping trust;
Output: F^t_{fc}: feature extraction model in the tth round;
eps: Deviation values for honest gradients;
μ: Standard
feature for honest gradients;

Step1. Obtain honest gradient training set using D_s:
Initialize the training set of the feature extraction model W^t_s;
for $i = 1$ **to** num **do**
  $w^t_{C_i} \leftarrow W^{t-1}$;
  for epoch $e = 1$ **to** E **do**
    $w^t_{C_i} \leftarrow w^t_{C_i} - \alpha \times \nabla l(b_s, w^t_{C_i})$; % b_s: batchsize
  end for
  $W^t_s[i] \leftarrow w^t_{C_i}$;
end for
Step2. Train local gradient feature extraction model:
Initialize feature extraction model F^t_{fc};
for epoch $e = 1$ **to** E_s **do**
  for $i = 1$ **to** num **do**
    $w^t_{C_i} \leftarrow W^t_s[i]$; $w^t_{C_i}.label = [1] \in \mathbb{R}^1$;
    $w^t_{C_i} \leftarrow$Flatten($w^t_{C_i}$);
    $F^t_{fc}.W \leftarrow F^t_{fc}.W - \beta \times \nabla J(w^t_{C_i}, F^t_{fc}.W)$;
  end for
end for
$eps \leftarrow \max F^t_{fc}(W^t_s) - \min F^t_{fc}(W^t_s)$;
$\mu \leftarrow \frac{1}{num}\sum_{i=1}^{num} F^t_{fc}(w^t_{C_i})$;
return F^t_{fc}, eps, μ;

The server constructs FEM for one class gradient as shown in Algorithm 1, which comprises two components: using D_s to obtain honest gradients for building FEM's training set, and using this set to train FEM. Specifically, we input D_s into the current global model W for training. The server obtains honest gradients w_{C_i} and assembles training set $W_s = \{w_{C_1}; \cdots; w_{C_{num}}\}$ for FEM. Training pairs $\{w_{C_i}, 1\}$, $i \in [1, num]$, are constructed to train FEM F_{fc}, where num denotes the number of samples in training set, $num = \frac{|D_s|}{|b_s|}$. F_{fc} training aims to obtain optimal network parameters w_i and b_i, $i \in [1, k]$,



(a) Normal circumstances (b) Malicious clients' proportion > 50%

Fig. 2. Aggregation results of Attack-adaptive Aggregation under different proportions of malicious attacks. Red represent malicious gradients, green represent honest gradients, and yellow represent the iterative aggregation estimates.

i.e., the resolution of the following optimization problem.

$$\text{argmin}_{w_i^*, b_i^*} \sum_{w \in W_s} J\left(\sigma\left(f_k \circ \ldots \circ f_2 \circ f_1(w)\right) - 1\right), \quad (1)$$

where f_k denotes the kth FCNN layer, J is the loss function, and σ denotes the activation function. The objective in training FEM is to push the predicted output value as close as possible to 1 for honest gradient inputs w. Once FEM F^t_{fc} is derived for one gradient class, deviation values eps and standard feature extraction output μ are computed for use in detecting malicious clients.

Local Gradient Feature Extraction: The server receives local gradients $\boldsymbol{w}^t = \{w^t_1; w^t_2; \cdots; w^t_n\}$ in the tth interaction. The extraction of local gradient features using FEM F^t_{fc} is shown in the Algorithm 2. In Algorithm 2, m denotes the number of local gradients classes, $w^t_j = \{v^t_{j1}, v^t_{j2}, \ldots, v^t_{jm}\}$, $j \in [1, n]$. Feature models $F^t_{fc_i}$, $i \in [1, m]$ are trained using Algorithm 1 for each class of local gradients. The server inputs the same class of local gradients $\boldsymbol{v}^t_i = \{v^t_{1i}; v^t_{2i}; \cdots; v^t_{ni}\}$ into the corresponding model $F^t_{fc_i}$ to obtain the predicted values $\boldsymbol{p}^t_i = \{p^t_{1i}; p^t_{2i}; \cdots; p^t_{ni}\} \in \mathbb{R}^{n*1}$. p^t_{ji} is the feature extraction value of the ith gradient of client j in the tth interaction. For each class of local gradients, the above process is used to obtain:

$$\begin{aligned} \boldsymbol{Pro}^t &= \{\boldsymbol{p}^t_1, \boldsymbol{p}^t_2, \ldots, \boldsymbol{p}^t_m\} \\ &= \{Pro^t_1; Pro^t_2; \cdots; Pro^t_n\} \in \mathbb{R}^{n*m}, \end{aligned} \quad (2)$$

where $Pro^t_j = \{p^t_{j1}, p^t_{j2}, \ldots, p^t_{jm}\} \in \mathbb{R}^{1*m}$ denotes m elementary feature vector extracted from the local gradients of client j in the tth interaction. Therefore, the server successfully transforms each client's m high-dimensional gradients into a $1 \times m$

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Local Gradient Feature Extraction.

Input: $\boldsymbol{F}^t_{\boldsymbol{fc}}$: Local gradient's FEM at the tth interaction, i.e., the output of Algorithm 1.
$\boldsymbol{F}^t_{\boldsymbol{fc}} = \{F^t_{fc_1}, F^t_{fc_2}, \ldots, F^t_{fc_m}\}$, where $F^t_{fc_i}$ denotes the FEM for the ith local gradients;
$\boldsymbol{w}^t$: Local gradients. $\boldsymbol{w}^t = \{w^t_1; w^t_2; \cdots; w^t_n\}$;
Output: $\boldsymbol{Pro}^t$: m elementary feature vector sets of all clients;
1: Split $\boldsymbol{w}^t$ into $\boldsymbol{v}^t_i$, $i \in \{1, 2, \ldots, m\}$;
2: **for** $i = 1$ **to** m **do**
3: $\quad \boldsymbol{p}^t_i \leftarrow F^t_{fc_i}(\boldsymbol{v}^t_i)$;
4: **end for**
5: $\boldsymbol{Pro}^t = \{\boldsymbol{p}^t_1, \boldsymbol{p}^t_2, \ldots, \boldsymbol{p}^t_m\} \in \mathbb{R}^{n*m}$;
6: **return** $\boldsymbol{Pro}^t = \{Pro^t_1; Pro^t_2; \cdots; Pro^t_n\}$;

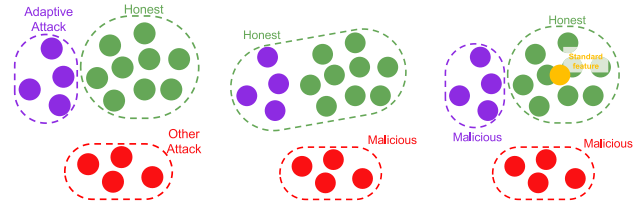
elementary feature vector using Algorithm 2. $\boldsymbol{Pro}^t \in \mathbb{R}^{n*m}$ represents the collection of m elementary feature vectors for all clients.

Remark: Comparison of Neural Network-Based Approaches: Attack-adaptive Aggregation [29] focuses on distinguishing between honest and malicious gradients in various attack scenarios by training the query encoder Q and key encoder K in the attention mechanism using malicious and honest gradients. In contrast, FLAD emphasizes training on honest data to extract features of honest gradients. Compared to FLAD, Attack-adaptive Aggregation has the following limitations:

1) *Pretraining Limitations:* Attack-adaptive Aggregation employs self-supervised learning to simulate various attack scenarios and collect malicious and honest gradients for training Q and K encoders. However, it is impossible to simulate all potential attack scenarios, which could lead to new attack methods rendering the pretrained Q and K ineffective.
2) *Aggregation Risks:* The core of Attack-adaptive Aggregation lies in adjusting the aggregation weights of gradients from different clients. However, there remains a risk of malicious gradients being aggregated into the global model parameters, even if their aggregation weights are very small, as shown in Fig. 2(a).
3) *Bias in Attention Mechanism:* When the number of malicious clients exceeds 50%, the initial estimation of the attention mechanism $q_0 = \text{med}(\{x_i\})$ may exhibit significant bias, which can result in the alignment scores shifting toward malicious gradients, leading to scenario in Fig. 2(b).

The goal of FLAD is to utilize neural networks to extract features of honest parameters and no attack samples are involved during training. FLAD focuses solely on identifying honest gradients, treating gradients that deviate from the honest features as malicious gradients, which makes FLAD applicable to a wide range of attack scenarios without requiring assumptions about the number of malicious clients, offering greater scalability and practicality.

2) Malicious Client Detection: FLAD uses dynamic clustering based on DBSCAN [36] and FEM for malicious clients detection, effectively identifying various attack types, even in scenarios where the proportion of malicious clients exceeds 50%. The representative clustering-based defences include Auror [33] and FLAME [8]. Compared with Auror, which uses K-means for binary classification, setting cluster number to two introduces the risk of misclassifying malicious gradients as honest. Regarding HDBSCAN-based FLAME, the absence of clustering range restriction makes it vulnerable to adaptive attack and unable to withstand scenarios where attackers exceed 50%. FLAD successfully identifies various attacks using DBSCAN, with adaptive standard feature μ and deviation value eps as the clustering range restriction, as shown in Fig. 3.



(a) Ground truth (b) Auror&FLAME (c) FLAD

Fig. 3. Comparison of different defences based on clustering, where standard feature in (c) FLAD correspond to the output of Algorithm 1.

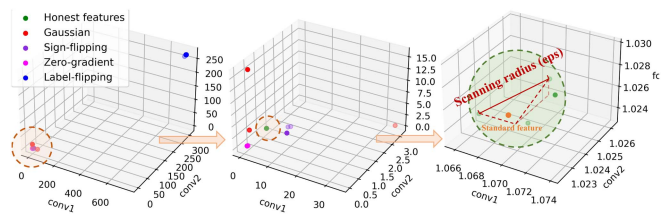


Fig. 4. DBSCAN separates honest and malicious gradient features, with dots representing feature extraction vectors for three classes of local gradients in the 1st interaction of MNIST (0.1) with CNN.

DBSCAN, a density-based clustering method, adapts its cluster number according to the specific data. DBSCAN is used to cluster $\boldsymbol{Pro}^t$ in FLAD, categorizing local gradients into honest and malicious with μ and eps. Fig. 4 illustrates DBSCAN's application in categorizing gradient feature vectors. Feature vectors extracted from honest gradients are highly similar and cluster within a sphere, defined by scan radius eps and μ. The key parameter scan radius in DBSCAN is related to the output of FEM in Algorithm 1. For m classes of model gradients used for detection, the joint deviation value $eps = ||\boldsymbol{eps}|| = \sqrt{\sum_{i=1}^{m} eps_i^2}$ is the scan radius of DBSCAN. Therefore, DBSCAN's scan radius is determined by server dataset for each interaction, which avoids false positives caused by artificial parameter settings.

Malicious client detection is described in Algorithm 3. The server may classify the local gradients feature vectors into many classes. Assuming clustering labels obtained from $\boldsymbol{Pro}^t$ are $label$, the server must identify the honest label Key from $label$. We use *cosine similarity* $\boldsymbol{Cos}$ and *relative length difference* $\boldsymbol{Len}$ to ascertain the honest label. We calculate Cos_l and Len_l

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3: Malicious Client Detection.

Input: $\boldsymbol{Pro}^t$: m elementary feature vector sets of all clients; $\boldsymbol{eps} = \{eps_1, \ldots, eps_m\}$: Deviation values for m honest gradients; $\boldsymbol{\mu} = \{\mu_1, \ldots, \mu_m\}$: Standard features for m honest gradients;
Output: $Malicious$: Malicious client list, $1 \sim n$;
$eps \leftarrow ||\boldsymbol{eps}||$; $\boldsymbol{Cos} \leftarrow$ None; $\boldsymbol{Len} \leftarrow$ None;
$label \in \mathbb{R}^{1*n} \leftarrow$ DBSCAN($\boldsymbol{Pro}^t, eps$);
for $j = 1$ **to** n **do**
  $l \leftarrow j_{label}$
  if $Cos_l ==$ None **then**
    $Cos_l \leftarrow cos(\boldsymbol{\mu}, Pro_j^t)$ % Cosine similarity
    $Len_l \leftarrow len(\boldsymbol{\mu}, Pro_j^t)$ % Relative length difference
  end if
end for
Take label Key in $\max[\alpha(Cos_l - Len_l) + (1-\alpha)||Pro_j^t||]$;
for $j = 1$ **to** n **do**
  if $j_{label} \neq Key$ **then**
    Add j to $Malicious$ list;
  end if
end for
return $Malicious$;

Algorithm 4: FLAD.

Input: W^{t-1}: Global model gradients at $t-1$th round; D_i: Local dataset for client i, $i \in \{1, 2, \ldots, n\}$; D_s: Server dataset for bootstrapping trust;
Output: W^t: Global model gradients after the tth aggregation;
Clinet side:
for i **to** n **in parallel do**
  $\boldsymbol{w_h^t} \leftarrow LocalTraining(W^{t-1}, D_i), i \in [1, n-c]$;
  $\boldsymbol{w_m^t} \leftarrow \boldsymbol{X}, i \in [n-c+1, n]$; % $\boldsymbol{X}$: arbitrary vectors
end for
Send $\boldsymbol{w}^t \leftarrow \{\boldsymbol{w_h^t}; \boldsymbol{w_m^t}\}$ to the server;
Server side: m class model gradients for detection
for $i = 1$ **to** m **do**
  $F_{fc_i}^t, eps_i, \mu_i \leftarrow FeatureExtractionModel(W^{t-1}, D_s)$;
end for
$\boldsymbol{F_{fc}^t} = \{F_{fc_1}^t, \ldots, F_{fc_m}^t\}$;
$\boldsymbol{Pro}^t \leftarrow LocalGradientFeature(\boldsymbol{F_{fc}^t}, \boldsymbol{w}^t)$;
$\boldsymbol{eps}, \boldsymbol{\mu} = \{eps_1, \ldots, eps_m\}, \{\mu_1, \ldots, \mu_m\}$;
$Malicious \leftarrow MaliciousClientDetection(\boldsymbol{Pro}^t, \boldsymbol{eps}, \boldsymbol{\mu})$;
$W^t \leftarrow Aggregation(Malicious, \boldsymbol{w}^t)$;
Send W^t to all clients;

between feature vectors Pro_j^t labeled l and standard feature $\boldsymbol{\mu}$:

$$\text{Cos}_l = \cos(\boldsymbol{\mu}, Pro_j^t) = \frac{< \boldsymbol{\mu}, Pro_j^t >}{||\boldsymbol{\mu}|| \cdot ||Pro_j^t||}, \tag{3}$$

$$Len_l = len(\boldsymbol{\mu}, Pro_j^t) = |\frac{||\boldsymbol{\mu}||}{||Pro_j^t||} - 1|, \tag{4}$$

where $\boldsymbol{\mu}$ is $\{\mu_1, \ldots, \mu_m\}$, which comprises the standard features of m classes of model gradients and is used as honesty standard. Considering that the target label for FEM training is 1, honest similarity of label l is defined as $S_l = \alpha(Cos_l - Len_l) + (1-\alpha)||Pro_j^t||$, $0 < \alpha < 1$. Honest label Key is determined by the maximum S, and remaining labels are categorized as malicious.

C. Complete FLAD Algorithm

Algorithm 4 presents the comprehensive Byzantine-robust FL approach FLAD, involving c malicious clients. The server uses FedAvg to aggregate honest gradients after rejecting malicious clients. With malicious clients excluded, FedAvg can effectively merge and learn local gradients from honest clients, facilitating rapid convergence and high accuracy for global model.

D. Proximity Analysis of Tolerance for Malicious Attacks

This section is used to analyze the proximity point of Byzantine attacks tolerated by FLAD. The training objective of FEM is to obtain optimal w_i^* and b_i^* in formula (1). We can obtain the feature extraction vectors for num server model gradients:

$$f_{c_i} = \sigma\left(f_k \circ \ldots \circ f_2 \circ f_1(w_{c_i})\right), i \in [1, num]. \tag{5}$$

DBSCAN's scan radius $eps = ||\max f_{c_i} - \min f_{c_i}||$, where $||\cdot||$ denotes the L_2 norm. The average value $\mu = \frac{1}{num}\sum_{i=1}^{num} f_{c_i}$ used to measure the standard of honest gradients. Local gradients from clients are fed into the FEM to obtain the corresponding feature extraction $F = \{f_1, \ldots, f_i, \ldots, f_n\}$.

Assumption 1: There are no less than three honest gradients in local gradients, and the client feature vector with the closest L_2 norm from the average value μ is f_q:

$$f_q = \text{argmin}_{f_i \in F} ||f_i - \mu||. \tag{6}$$

So f_q is considered as the feature extraction vector of the honest client q and w_q is the honest local gradient.

Since the feature f_q corresponding to w_q is closest to the features of honest gradient μ, it is reasonable to assume that w_q from client q is the honest gradient.

Definition 1: There exists vectors set $F = \{f_1, f_2 \cdots\}$. If there exists no less than three elements $\{f_y, \cdots\}$ in F within eps L_2 norm of vector f_x, f_y is defined as $1eps - f_x$ directly connected point. If there are also no less than three elements $\{f_z, \cdots\}$ in F within eps L_2 norm of vector f_y, f_z is defined as $2eps - f_x$ directly connected point, and so on.

Feature extraction vectors set from local gradients is $\{f_1, \cdots f_q, \ldots, f_n\}$, and we assume that there exist at most k$eps - f_q$ directly connected point. According to the principle of DBSCAN, feature vector f_l with the farthest L_2 norm from f_q and reachable density with f_q has $||f_l - f_q|| \leq$ keps. The criticality of the honest and malicious gradients has the following Theorem 1.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

Theorem 1: A sufficient condition for the malicious gradient feature f_m to be successfully detected is $||f_m - f_q|| \geq (k+1)eps \geq ||f_l - f_q|| + eps$. A necessary condition for the honest gradient feature f_h to be successfully identified is $||f_h - f_q|| \leq keps$.

E. Convergence Analysis of FLAD

We will prove the convergence of the FLAD in this section.

Assumption 2 ((Error Term).): There exists an error term $\boldsymbol{\epsilon}$ between the malicious gradients and the honest gradients, such that the equation holds: $\sum_{x\in M} \boldsymbol{w}'_{\boldsymbol{x}} = \sum_{x\in H} \boldsymbol{w}_{\boldsymbol{x}} + \boldsymbol{\epsilon}$, where M and H denote the sets of malicious and honest clients, respectively. Specifically, $H = [1, n-c]$ and $M = [n-c+1, n]$.

Proof: The process of poisoning attack is to find a set of malicious model gradients $\boldsymbol{w}'$ which are different from the honest client's target model $\boldsymbol{w}$, i.e. $\boldsymbol{w} \neq \boldsymbol{w}'$. Let the distance between $\boldsymbol{w}'$ and $\boldsymbol{w}$ be $\overline{\boldsymbol{h}}$, i.e. $\overline{\boldsymbol{h}} = \boldsymbol{w} - \boldsymbol{w}'$. As described in Section III-A, malicious and honest clients can only achieve their goals by submitting gradients.

We define the classical poisoning attack model. Suppose that the gradients $\{\boldsymbol{w}_{\boldsymbol{x}}, x \in H\}$ of honest clients in round t are independent and identically distributed samples taken from the random variable $\boldsymbol{w} \leftarrow \boldsymbol{w} - \alpha \times \nabla l(b_s, \boldsymbol{w})$, where $E[\boldsymbol{w}] = \overline{\boldsymbol{w}}$ is an unbiased estimate of the gradient. Thus, for any $x \in H$, we have $E[\boldsymbol{w}_{\boldsymbol{x}}] = E[\boldsymbol{w}] = \overline{\boldsymbol{w}}$. In the presence of malicious clients, the malicious gradients received by server are $\{\boldsymbol{w}'_{\boldsymbol{x}}, x \in M\}$ as follows:

$$\boldsymbol{w}^{\boldsymbol{\tau}}_{\boldsymbol{x}} = \begin{cases} \boldsymbol{w}_{\boldsymbol{x}}, & \text{if the } x\text{-th client is honest,} \\ \boldsymbol{w}'_{\boldsymbol{x}}, & \text{if the } x\text{-th client is malicious.} \end{cases} \tag{7}$$

An important assumption for convergence of the SGD algorithm is that each honest gradient is an unbiased estimate of actual gradient, which is usually ensured by uniformly randomly sampling the batch data b_s. However, the gradients submitted by malicious clients violate the principle of uniform random sampling, making malicious gradients significantly different from honest ones. According to the definition of SGD algorithm, N is the set of all clients, $N \in [1, n]$:

$$\boldsymbol{w}^{\boldsymbol{\tau}} \leftarrow \frac{1}{|N|} \sum_{x\in N} \boldsymbol{w}^{\boldsymbol{\tau}}_{\boldsymbol{x}}, \tag{8}$$

The goal of malicious gradients $\{\boldsymbol{w}'_{\boldsymbol{x}}, x \in M\}$ is to obtain parameters:

$$\boldsymbol{w}' \leftarrow \frac{1}{|M|} \sum_{x\in M} \boldsymbol{w}'_{\boldsymbol{x}}, \tag{9}$$

Similarly, honest gradients $\{\boldsymbol{w}_{\boldsymbol{x}}, x \in H\}$ is to obtain parameters:

$$\boldsymbol{w} \leftarrow \frac{1}{|H|} \sum_{x\in H} \boldsymbol{w}_{\boldsymbol{x}}, \tag{10}$$

Based on the previously defined parameter distance ($\overline{\boldsymbol{h}} = \boldsymbol{w} - \boldsymbol{w}'$), we have:

$$\begin{aligned} \sum_{x\in M} \boldsymbol{w}'_{\boldsymbol{x}} &= \frac{|M|}{|H|} \sum_{x\in H} \boldsymbol{w}_{\boldsymbol{x}} - |M|\overline{\boldsymbol{h}} \\ &= \left(\frac{|M|}{|H|} - 1\right) \sum_{x\in H} \boldsymbol{w}_{\boldsymbol{x}} - |M|\overline{\boldsymbol{h}} + \sum_{x\in H} \boldsymbol{w}_{\boldsymbol{x}} \\ &= \left(\frac{|M|}{|H|} - 1\right) |H|\overline{\boldsymbol{w}} - |M|\overline{\boldsymbol{h}} + \sum_{x\in H} \boldsymbol{w}_{\boldsymbol{x}}, \end{aligned} \tag{11}$$

Thus, the error term $\boldsymbol{\epsilon}$ between malicious gradient and honest gradient satisfies the following relationship:

$$\boldsymbol{\epsilon} = \left(\frac{|M|}{|H|} - 1\right) |H|\overline{\boldsymbol{w}} - |M|\overline{\boldsymbol{h}}. \tag{12}$$

Proposition 1 ((Convergence).): According to the detailed steps of FLAD in Section IV, the convergence speed of malicious and/or honest clients is $O(\frac{1}{T^2})$, where T is the number of iterations.

Proof: After identifying honest clients, FLAD employs the traditional FedAvg aggregation, which follows the basic framework of adaptive learning rate method [37] which has been applied to SGD algorithm and provides convergence guarantees. SGD algorithms with constant learning rate have been proved to converge at a rate of $O(1/T^2)$.

Assuming an equal amount of local data for each client, the FedAvg aggregation weight of user U_x is determined by function $f(x,t)$ as $\frac{1}{|M^t|+|H^t|}$, $x \in \{M^t, H^t\}$ or 0, $x \notin \{M^t, H^t\}$, where t denotes the current number of iterations, and M^t and H^t denote the clients that are recognized as honest in iteration t. U_x may be either a malicious or an honest client, $U_x \in M$ or H. In Section III, we assume that the number of honest clients is more than 2, i.e. $|H| > 2$, to ensure that correct data exists in federated training. The convergence property of SGD can be applied to FLAD as long as the training is based on data of honest clients. At this point, FLAD needs to satisfy two conditions:

- *Condition 1:* for all $x \in M$, $f(x,t) \to 0$,
- *Condition 2:* for all $x \in H$, $f(x,t) \to 1$.

We consider $\boldsymbol{w}^*_{\boldsymbol{x}}$ as the ideal gradient from initial model $\boldsymbol{w}_{\boldsymbol{init}}$ to optimized model with respect to local data of U_x. According to Assumption 2, $\sum_{x\in M} \boldsymbol{w}^*_{\boldsymbol{x}} = \sum_{x\in H} \boldsymbol{w}^*_{\boldsymbol{x}} + \boldsymbol{\epsilon}$. FLAD can identify anomalous gradients based on differences and reject malicious gradients. As the number of iterations increases, the target of malicious gradients deviate further away from honest gradients and the error term $\boldsymbol{\epsilon}$ will increase. Thus malicious clients $x \in M$ can be more obviously recognized and rejected by FLAD, which will result in malicious gradients tending to have a weight of 0 when globally aggregated, i.e. $|M^t| \to 0$, $|H^t| \to |H|$, $f(x,t) \to 0$. Therefore, malicious gradients have a weight of 0, which satisfies Condition 1. Accordingly, FLAD satisfies for honest clients:

$$\lim_{t\to\infty} \frac{|H|}{|M^t| + |H^t|} = 1. \tag{13}$$

Therefore, the sum of weights of honest clients converges to 1, which satisfies Condition 2. The above proof shows that FLAD satisfies both conditions. Therefore, the convergence rate of FLAD is consistent with that of SGD with adaptive learning rate, i.e. $O\left(\frac{1}{T^2}\right)$.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DATASET AND GLOBAL MODEL SETTINGS

DATASET	CLIENT NUMBER	CLINET DATASET SIZE	FEATURES/QUESTIONS	GOBAL MODEL	BATCHSIZE	SERVER DATASET
MNIST	50	1200	784 (28*28)	2 CNN+1 FC	64	300
CIFAR-10	50	1000	3072 (3*32*32)	RESNET-18	64	200
ASSIST2009	20	1.6K	26.7K	MAML(2FC)	128	500
FEMNIST	3400 (50 PER ROUND)	2-582 (AVERAGE:226.83)	784 (28*28)	VGG-16	64	300

TABLE II
CNN STRUCTURE FOR TRAINING MNIST

LAYER	SIZE
INPUT	1×28×28
CONVOLUTION+RELU	10×1×5×5
MAX POOLING	2×2
CONVOLUTION+RELU	20×10×5×5
MAX POOLING	2×2
LINEAR+SOFTMAX	320×10
OUTPUT	10

V. PERFORMANCE EVALUATION

A. Experiment Setup

1) Datasets and FL System Settings: We have experimented with four datasets: MNIST [38], CIFAR-10 [39], ASSIST2009[2] and FEMNIST [40]. The information about the four datasets and their corresponding global model gradients are shown in Table I, and the specific global model architecture for MNIST is shown in Table II. Following the work [14], in each interaction, the server selects all clients' local gradients for aggregation, except for FEMNIST. The client interacts with the server a total of 20 times in MNIST, CIFAR-10 and FEMNIST and 5 times in ASSIST2009.

Non-IID and IID data: For image dataset, we define distribution probability q to measure the distribution difference among different client dataset. A larger q indicates a higher degree of Non-IID. Refer to experimental section in FLTrust [6] for detailed definitions of q. We choose $q = 0.1,\ 0.8$ for MNIST and $q = 0.1,\ 0.5$ for CIFAR-10. If $q = 0.1$, the client's data is IID. ASSIST2009 dataset is derived from interactions by different students answering different questions, making it inherently Non-IID.

Server dataset: In order to highlight the robustness to D_s, we borrow the experimental strategy of FLTrust [6] and simulate D_s for the following two scenarios to explore their impact on global accuracy of FLAD.

- *Case I:* D_s has the same distribution as training dataset.
- *Case II:* The distribution of D_s differs from the training dataset. Similar to FLTrust [6], we measure the deviation of D_s from the training dataset distribution with bias probability P. A higher P corresponds to a greater disparity between D_s and the training dataset. In our experiments, bias labels are assigned as "0" for MNIST and FEMNIST, and as "airplane" for CIFAR-10.

FLAD Settings: For MNIST, two convolutional kernels and one fully connected weights are used to construct FEM. For CIFAR-10, we select the convolutional kernels of the first convolutional layer in conv1 and conv2 modules and the fully connected weight of the last layer. For ASSIST2009, we select two fully connected weights. For FEMNIST, we select the fully connected weights of the last three layers. For FEM's training, the loss function J chooses MSE, the optimizer is Adam, the learning rate is $\beta = 1 \times 10^{-3}$, and the total number of training epoch is $E_s = 10$.

2) Evaluated Defences: We choose robust aggregation (Krum [22], Bulyan [24], Median [23]), correction of gradients (FLTrust [6]) and malicious node detection and rejection (RWD [34], FLAME [8], FLDetector[3] [32]) for seven defences. Moreover, we use FedAvg, which is dominant in the non-adversarial setting, as the experiment baseline. The threshold in RWD is $T = 10$ and the size of FLTrust root dataset is the same as FLAD. Adaptive noising level in FLAME is $\boldsymbol{\sigma} = 0.001$. The remaining parameter settings are the same as in the experimental part of the original paper.

3) Evaluated Attacks: For untargeted attacks, we choose Gaussian attack [10], sign-flipping attack [12], zero-gradient attack [11], adaptive model poisoning attack [14], AGR-agnostic attack [15] and MPAF [13]. For targeted attacks, we choose backdoor data poisoning, model-replacement attack [18] and DBA [20]. Furthermore, we assume that these attacks are implemented in the context of full knowledge. Gaussian, sign-flipping and zero-gradient attacks have the same settings as [11].

Adaptive Local Model Poisoning Attack: We use the current state-of-the-art adaptive attack framework [14] to design adaptive attack to explore the tolerance proximity of FLAD for malicious poisoning attacks. The adaptive attack framework is as follows.

$$\begin{aligned} & \max_{w'_1,\ldots,w'_c} \mathbf{s}^T \left(W - W'\right), \\ \text{subject to} \quad & W = \mathcal{A}\left(w_1, \ldots, w_c, w_{c+1}, \ldots, w_n\right), \\ & W' = \mathcal{A}\left(w'_1, \ldots, w'_c, w_{c+1}, \ldots, w_n\right), \end{aligned} \tag{14}$$

where the first c clients are assumed to be malicious. w'_i is the poisoned local gradients uploaded by the ith malicious client. $\mathbf{s}$ is the column vector of the direction of change of the global gradients without attack. W and W'_i represent the global model obtained with aggregation rule $\mathcal{A}$ under poisoning and no attack, respectively. In summary, the adaptive attack treats the attack as an optimization problem in which the attackers dynamically optimize the malicious local gradients in each iteration, such that the global model gradient after aggregation changes maximally in the opposite direction as it would have done without attack. Different aggregation rules will lead to different optimization problems. Following the above adaptive local poisoning attack

[2]https://sites.google.com/site/assistmentsdata/home/assistment-2009-2010-data

[3]FLDetector is abbreviated as FLD in our experiments.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

framework, we define W and W':

$$W = \mathcal{A}(w_1, w_2, \ldots, w_{n-c}), \tag{15}$$

$$W' = \mathcal{A}(w_1, w_2, \ldots, w_{n-c}, w'_{n-c+1}, \ldots, w'_n). \tag{16}$$

The adaptive model attack can be considered as the optimization problem $\max_{w'_{n-c+1},\ldots,w'_n} \mathbf{s}^T(W - W')$. We assume that the number of local data for each client is equal. Since the values of honest gradients do not affect the results of problem. Therefore, the optimization can be simplified as:

$$\min_{w'_{n-c+1},\ldots,w'_n} \mathbf{s}^T \left(w'_{n-c+1} + \cdots + w'_n\right). \tag{17}$$

Therefore, attacker's goal is to carefully construct malicious gradients w' in the opposite direction of global update without attacks as much as possible to confuse FEM in anomaly detectors so that FEM outputs the wrong feature extraction vectors for the malicious gradients, misleading the Algorithm 3 to classify them as honest gradients. The solution to the optimization problem translates into generating holistic confused samples of FEM based on deviating the honest gradients as much as possible.

We borrow FGSM (Fast Gradient Sign Method) [41] to generate adversarial samples to solve the problem (17) and obtain adaptive attack of FLAD in Algorithm 5. We consider all malicious gradients as a whole and jointly compute their updated gradients on FEM. Honest gradient w_i added to noise u is used as initial value for malicious gradient w'_j, avoiding the problem that initial w'_j has a loss function of 0, where u obeys a multivariate Gaussian distribution $\mathcal{N}(0, \sigma^2\mathbf{I})$. We use Algorithm 2 to obtain the feature vectors Pro'_j corresponding to the current w'_j, and take the difference between Pro'_j and Pro_i as the loss to compute the updated gradient $\nabla loss(w'_j)$ of w'_j in FEM. The new optimized w'_j is obtained with ϵ as step size, which is 1e-7in experiments. When w'_j deviates too much to be successfully detected by Algorithm 3, the optimization result B'_j from the previous iteration is returned as the optimal adaptive malicious gradient w'_j.

AGR-agnostic Attack: The research [15] introduced two types of attacks: AGR-tailored attacks targeting specific aggregation algorithms (e.g., Krum, Bulyan) and AGR-agnostic attacks that do not depend on the aggregation algorithm. The core idea of AGR-tailored attacks is similar to the Adaptive Local Model Poisoning Attack [14], and there are no specific attack strategies directly targeting FLAD. Therefore, we select the more challenging-to-detect Min-Sum attack strategy from the AGR-agnostic attacks described in [15]. The specific parameter settings are consistent with [15].

Model Poisoning Attack based on Fake Clients (MPAF): The attacker injects multiple fake clients, which participate in training and send malicious local updates. The goal of these malicious updates is to steer the global model's parameters toward a low-performance model chosen by the attacker, thereby significantly reducing the accuracy of the global model [13]. To highlight the superiority of FLAD, we set the proportion of injected fake clients to 20% (10 fake clients in MNIST, CIFAR-10 and FEMNIST), the sample rate $\beta = 1$, the scaling factor $\lambda = 1 \times 10^3$, and the base model w' as a randomly initialized model.

Algorithm 5: Adaptive Local Model Attack on FLAD.

Input: w_i: Local gradients sent by honest clients, $i \in [1, n-c]$; F_{fc}: Feature extraction model; ϵ: Step size of optimization per iteration;
Output: w'_j: Malicious gradients carefully constructed by the malicious clients, $j \in [n-c+1, n]$;

```
Pro_i ← LocalGradientFeature(F_fc, w_i);
Initialize w'_j, Pro'_j ← w_i, Pro_i;
w'_j ← w'_j + ε × u, u ~ N(0, σ²I);
while True do
    B'_j ← w'_j;
    Pro'_j ← LocalGradientFeature(F_fc, w'_j);
    loss ← MSE(Pro'_j, Pro_i);
    Update w'_j ← w'_j + ε × sgn(∇loss(w'_j));
    w = {w_i, w'_j};
    if F_fc(w) is able to detect malicious clients then
        Break;
    end if
end while
return w'_j ← B'_j;
```

Backdoor Data Poisoning Attack: Backdoor data poisoning attack requires the attacker to implant the backdoor trigger in the source training data so that the final model will function properly on clean data, but will constantly predict the target class chosen by the attacker when data containing the trigger is present. In our experiments, 80% of the backdoor data is injected into local dataset of the backdoor client during the training and we inject 8×8 square pixel block in the top left corner of the malicious client local dataset as backdoor trigger. For MNIST and FEMNIST, we change the target label of the image with 8×8 black block to "0". For CIFAR-10, we inject pixel blocks corresponding to the channel color on three channels and change the target label to "airplane". After injecting the backdoor to local dataset and changing the target label, malicious client train local model as an honest client.

Model-replacement Attack: The backdoor trigger of model-replacement attack is set up as for backdoor data poisoning attack. Following the work [18], in order to mitigate the dilution of the attack effect due to global aggregation, we extend the poisoning local gradients by setting the weight factor $\gamma = n$ and send the extended local gradients to the server [18], where n is the total number of clients.

Distributed Backdoor Attack (DBA): We split the 8×8 centralized trigger into four 2×8 local triggers and evenly divide malicious clients into four groups to inject different local triggers. During training, the server will receive local gradients containing local triggers from four groups of malicious clients. The complete algorithm is detailed in DBA [20].

B. Experiment Results

1) Defense Goals: We define four defensive goals of accuracy, robustness, efficiency and universality in Section V-B1.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

TABLE III
GLOBAL MODEL ACCURACY FROM DIFFERENT DEFENSES UNDER DIFFERENT ATTACKS

(a) MNIST (0.8, 20%)

ATTACKS	FEDAVG	KRUM	BULYAN	FLTRUST	FLAME	FLD	FLAD
NONE	94.9	90.2	94.3	94.7	70.5	95.1	95.2
GAUSSIAN	9.1	91.1	94.6	95.0	43.4	94.3	95.2
SIGN-FLIPPING	31.4	90.6	94.7	94.8	57.3	94.0	95.0
ZERO-GRADIENT	10.1	91.6	94.1	94.7	64.7	93.8	95.1
AGR-AGNOSTIC	27.4	87.5	91.6	92.6	33.4	93.3	94.5
MPAF	19.8	88.2	92.7	92.9	45.3	93.4	94.7
BACKDOOR-DATA	94.7	91.2	94.2	93.6	72.4	94.1	95.0
MODEL-REPLACE	94.3	89.4	93.8	93.7	70.8	94.1	94.9
DBA	94.3	87.1	94.1	93.5	53.8	94	94.7
AVERAGE	52.9	89.7	93.8	93.9	56.8	94.0	**94.9**

(b) CIFAR-10 (0.1, 20%)

ATTACKS	FEDAVG	KRUM	BULYAN	FLTRUST	FLAME	FLD	FLAD
NONE	68.8	53.5	64.1	54.4	47.4	68.3	68.6
GAUSSIAN	14.3	62.3	63.1	53.3	9.9	67.5	69.0
SIGN-FLIPPING	10.0	54.6	63.9	54.6	11.8	67.3	68.2
ZERO-GRADIENT	10.0	50.7	62.0	53.8	10.9	66.8	69.3
AGR-AGNOSTIC	11.2	46.9	59.6	50.2	19.8	65.2	66.7
MPAF	10.1	49.1	60.8	52.7	23.5	65.9	67.2
BACKDOOR-DATA	67.2	51.7	62.3	53.6	45.7	67.6	69.1
MODEL-REPLACE	68.2	49.9	61.0	51.6	46.7	67.9	69.4
DBA	68.8	51.8	63.7	55.4	46.1	68.1	69.0
AVERAGE	36.5	52.3	62.3	53.3	29.1	67.2	**68.5**

(c) CIFAR-10 (0.5, 20%)

ATTACKS	FEDAVG	KRUM	BULYAN	FLTRUST	FLAME	FLD	FLAD
NONE	64.8	29.0	60.7	58.0	41.5	64.2	64.6
GAUSSIAN	14.1	46.7	62.6	55.2	9.1	63.4	64.1
SIGN-FLIPPING	10.0	31.0	62.3	56.8	11.2	63.0	63.9
ZERO-GRADIENT	12.1	29.5	62.0	55.8	9.7	63.3	64.1
AGR-AGNOSTIC	9.6	42.1	62.1	54.2	8.6	63.2	64.0
MPAF	11.3	42.2	63.5	56.7	10.0	63.4	64.1
BACKDOOR-DATA	64.0	33.9	61.3	50.8	40.8	64.1	64.2
MODEL-REPLACE	64.4	24.3	60.7	51.0	39.9	63.6	63.9
DBA	65.0	40.2	61.7	51.4	41.1	63.1	64.1
AVERAGE	35.0	35.4	61.9	54.4	23.5	63.5	**64.1**

(d) FEMNIST (20%)

ATTACKS	FEDAVG	KRUM	BULYAN	FLTRUST	FLAME	FLD	FLAD
NONE	77.4	74.5	76.1	76.7	58.5	77.1	77.4
GAUSSIAN	11.2	75.6	77.0	77.1	31.9	76.5	77.2
SIGN-FLIPPING	15.7	74.9	76.8	76.5	42.8	76.3	76.9
ZERO-GRADIENT	9.8	73.6	75.4	76.9	50.3	75.9	77.0
AGR-AGNOSTIC	12.8	70.7	73.5	73.8	28.1	74.9	75.8
MPAF	17.4	72.0	72.8	74.5	40.3	75	76.6
BACKDOOR-DATA	75.4	72.8	76.1	76.3	62.3	76.1	77.0
MODEL-REPLACE	75.3	73.3	76.7	76.0	60.7	76.1	77.1
DBA	76.2	70.1	76.9	75.4	49.8	76.4	77.0
AVERAGE	41.2	73.1	75.7	75.9	47.2	76.0	**76.9**

(e) ASSIST2009 (20%)

ATTACKS	FEDAVG	KRUM	BULYAN	FLTRUST	FLAME	FLD	FLAD
NONE	65.4	63.7	63.0	63.6	64.5	64.4	65.1
GAUSSIAN	50.9	63.3	63.0	57.3	63.5	64.1	65.0
SIGN-FLIPPING	60.1	63.1	64.1	60.3	63.5	64.6	65.4
ZERO-GRADIENT	60.1	63.1	64.1	60.3	63.8	64.7	65.4
AVERAGE	59.2	63.3	63.5	60.4	63.8	64.5	**65.3**

Percentages indicate the percentage of malicious clients and % are omitted.

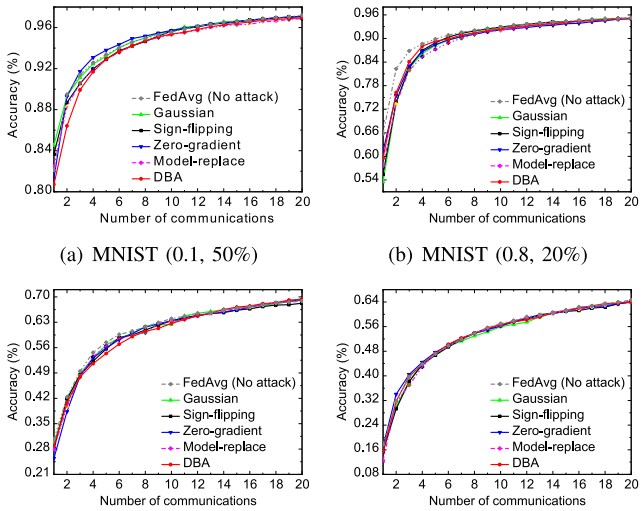
TABLE IV
BACKDOOR MODEL ACCURACY FROM DIFFERENT DEFENSES UNDER DIFFERENT TARGETED ATTACKS

(a) MNIST (0.1, 50%)

ATTACKS	FEDAVG	MEDIAN	KRUM	FLTRUST	FLAME	FLD	FLAD
NONE	10.0	10.1	10.1	10.0	10.0	10.0	10.0
BACKDOOR-DATA	99.9	99.1	27.6	10.3	10.1	10.1	9.9
MODEL-REPLACE	83.7	54.2	0.0	10.1	9.9	10.0	9.8
DBA	100.0	63.0	100.0	10.2	10.5	10.1	10.0
AVERAGE	73.4	56.6	34.4	10.2	10.1	10.1	**9.9**

(b) CIFAR-10 (0.1, 20%)

ATTACKS	FEDAVG	MEDIAN	KRUM	FLTRUST	FLAME	FLD	FLAD
NONE	10.0	9.9	10.2	10.0	10.1	10.0	10.0
BACKDOOR-DATA	57.2	10.2	10.0	11.0	10.2	10.4	11.0
MODEL-REPLACE	58.0	9.6	10.5	10.8	9.8	9.9	10.3
DBA	59.2	10.1	9.9	12.0	10.0	10.1	9.6
AVERAGE	46.1	10.0	10.2	11.0	10.0	10.1	**10.0**



(a) MNIST (0.1, 50%) (b) MNIST (0.8, 20%) (c) CIFAR-10 (0.1, 20%) (d) CIFAR-10 (0.5, 20%)

Fig. 5. Global accuracy of FLAD under different attacks.

The extensive experimental results show that our FLAD achieves these goals.

Accuracy: In the non-adversarial setting, FLAD's global accuracy is similar to FedAvg in Table III. On the MNIST (0.8), FedAvg, FLD and FLAD both achieve an accuracy of 95%, while Krum and Bulyan achieve 90% and 94% respectively. On the CIFAR-10 (0.1), the accuracy of FLAD and FedAvg can reach 69%, while the accuracy of FLTrust and FLAME are 54% and 47% respectively. Therefore, in the non-adversarial setting, FLAD exhibits higher global accuracy.

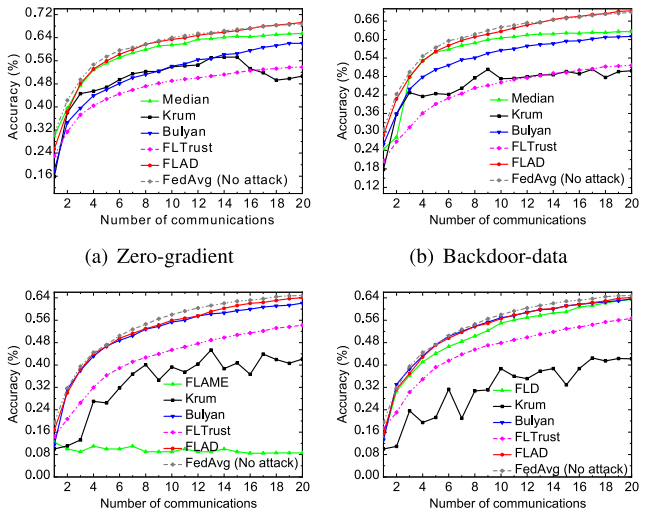
Robustness: The global accuracy of FLAD under different attacks is at most 2.1% lower than FedAvg without attack in Table III. However, other FL defences show reduced global accuracy in some attack scenarios. For example, the accuracy of Bulyan, FLAME and FLD can be up to 8.4%, 59.1% and 2.5% lower than FLAD respectively in Table III(b). Moreover, FLAD safeguards against targeted attacks while maintaining global accuracy, where FLAD's backdoor accuracy is maintained at the same 10% as when there are no attacks in Table IV.

Efficiency: FLAD only requires client to train its own local model in each iteration, does not introduce additional computational overhead for clients and does not add additional communication overhead between client and server compared to FedAvg. As shown in Fig. 5, the convergence speed of FLAD is comparable to FedAvg without attacks under different attacks. However, other defences converge significantly slower than the

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

TABLE V
DETECTION ACCURACY IN RWD, FLAME, FLD AND FLAD, WHERE *FPR* IS THE RATE OF HONEST NODES MISCLASSIFIED AS MALICIOUS, *FNR* IS THE RATE OF MALICIOUS NODES MISCLASSIFIED AS HONEST, *RECALL* IS THE RATE OF MALICIOUS NODES CORRECTLY DETECTED, AND *PRECISION (PRE)* IS THE RATE OF PREDICTED MALICIOUS NODES THAT ARE ACTUALLY MALICIOUS

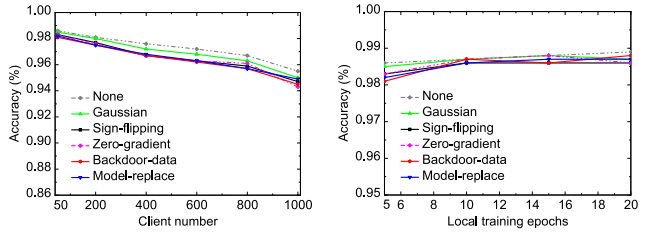
ATTACKS	RWD				FLAME				FLD				FLAD			
	FPR	FNR	RECALL	PRE	FPR	FNR	RECALL	PRE	FPR	FNR	RECALL	PRE	FPR	FNR	RECALL	PRE
GAUSSIAN	0.0	0.0	100.0	100.0	63.1	27.5	72.5	27.7	6.5	0.0	100.0	92.9	1.3	0.0	100.0	96.2
SIGN-FLIPPING	100.0	100.0	0.0	0.0	59.2	18.0	82.0	31.6	4.0	0.0	100.0	95.6	0.5	0.0	100.0	98.5
ZERO-GRADIENT	100.0	100.0	0.0	0.0	67.9	13.5	86.5	29.8	4.4	0.0	100.0	94.8	0.3	0.0	100.0	99.0
BACKDOOR-DATA	100.0	100.0	0.0	0.0	83.6	30.5	69.5	21.7	2.1	0.0	100.0	97.1	0.5	0.0	100.0	98.5
MODEL-REPLACE	0.0	0.0	100.0	100.0	65.3	46.0	54.0	21.6	5.3	0.0	100.0	93.7	0.3	0.0	100.0	99.0
DBA	100.0	100.0	0.0	0.0	75.7	19.0	81.0	26.3	7.8	0.0	100.0	90.3	1.7	0.0	100.0	95.2
AVERAGE	66.7	66.7	33.3	33.3	69.1	25.8	74.3	26.5	5.0	0.0	100.0	94.1	**0.8**	**0.0**	**100.0**	**97.7**



(a) Zero-gradient (b) Backdoor-data

(c) AGR-agnostic (d) MPAF

Fig. 6. Global accuracy versus interaction number for different defences against different attacks in CIFAR-10(0.1, 20%) (a) and (b), CIFAR-10(0.5, 20%) (c) and (d).



(a) Clients (b) Local training epochs

Fig. 7. Global model accuracy with different numbers of clients/local training epochs under different attacks on MNIST (0.1, 20%).

no-attack FedAvg as shown in Fig. 6, which shows that FLAD global model converges faster than other FL defences, more closely matching no-attack FedAvg.

Generalizability: FLAD has been experimentally demonstrated to be applicable across various scenarios, including different numbers of clients and varying local training epochs. As shown in Fig. 7(a), when the number of clients increases from 50 to 1000, only a marginal decrease in accuracy is observed, primarily due to the increasing dispersion of data. The overall accuracy improves with an increase in the number of local training epochs as shown in Fig. 7(b). Furthermore, FLAD is capable of maintaining a high global accuracy even when the proportion of malicious clients reaches 50%, as discussed in Section V-B4.

2) Malicious Clients Detection Accuracy: The defence idea of FLAD is to detect and reject malicious gradients and to aggregate only honest local gradients. RWD [34], FLAME [8] and FLD [32] also aim at detecting and rejecting malicious gradients. Table V shows average malicious detection results during client-server interactions in MNIST (0.1, 20%). As shown in Table V, RWD works only in Gaussian and model-replacement attack and FLAME is not capable of identifying malicious nodes completely. Although both FLD and FLAD achieve 100% recall, FLAD demonstrates higher precision and lower FPR. Although FLAD may have cases of false positives, it can maintain a relatively high precision in our experiments. FLAD can successfully resist attacks, outperforming RWD, FLAME and FLD significantly.

3) Comparing Different Variants of FLAD: As shown in Fig. 1, the FLAD has three key components: server dataset, Feature Extraction Model, and DBSCAN clustering. According to the usage characteristics of each component, four variants of FLAD are considered for ablation experiments.

- *FLAD-onlyServer:* Only the server dataset is utilized for training to evaluate the performance of the server model, i.e., direct aggregation W_s^t in Algorithm 1.
- *FLAD-withServer:* The server model gradients W_s are aggregated along with the honest gradients, i.e. add the model of server dataset for aggregation.
- *FLAD-NoDBSCAN:* $\boldsymbol{\mu}$, $\boldsymbol{eps}$ is the output of Algorithm 1. If the feature extraction value of local gradient is in $[\boldsymbol{\mu} - \boldsymbol{eps}, \boldsymbol{\mu} + \boldsymbol{eps}]$, it is honest gradient, otherwise it is malicious gradient.
- *FLAD-NoFCNN:* The server does not use Feature Extraction Model and directly adopts DBSCAN to cluster. The parameter eps is the maximum value of L_2 norm between any two server model gradients.

The results in Table VI demonstrate that FLAD outperforms the four variants. Due to the limited size of the server dataset and the fewer training epochs of the server model compared to the local models on clients, both the model obtained by aggregating the server gradients with honest gradients in FLAD-withServer and the global model trained solely on the server dataset in FLAD-onlyServer experience a decline in accuracy. FLAD-NoDBSCAN direct use of range partitioning may ignore a part of honest local gradients due to data heterogeneity, as

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
EXPERIMENTAL RESULTS OF DIFFERENT FLAD VARIANTS UNDER DIFFERENT ATTACKS IN MNIST (0.8, 20%)

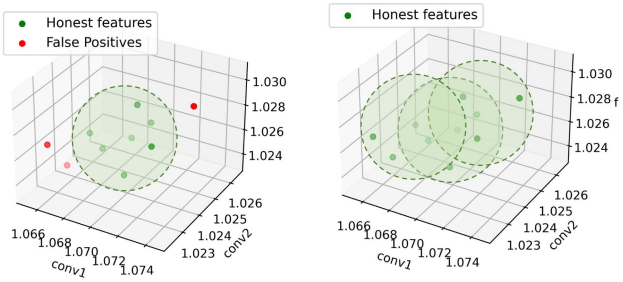
VARIANTS	NO ATTACK		GAUSSIAN		SIGN-FLIPPING		ZERO-GRADIENT		AGR-AGNOSTIC		MPAF		AVERAGE	
	ACC	DIV	ACC	DIV	ACC	DIV	ACC	DIV	ACC	DIV	ACC	DIV	ACC	DIV
FLAD-ONLYSERVER	72	-	-	-	-	-	-	-	-	-	-	-	72	-
FLAD-WITHSERVER	94	100	94	100	95	100	95	100	93	98	95	100	94	100
FLAD-NODBSCAN	94	90	94	87	32	9	93	94	95	78	94	58	84	69
FLAD-NOFCNN	46	9	38	30	56	29	65	30	92	49	94	50	65	33
FLAD	95	100	95	100	95	100	95	100	95	100	95	100	95	100

The results are presented as global accuracy (ACC) and correct division rate (Div) which indicates the ratio of nodes correctly divided among all nodes.

TABLE VII
THE TIME (IN SECONDS) REQUIRED FOR THE SERVER TO AGGREGATE THE GLOBAL GRADIENT PER ROUND UNDER MPAF ATTACK IN DIFFERENT DEFENSES

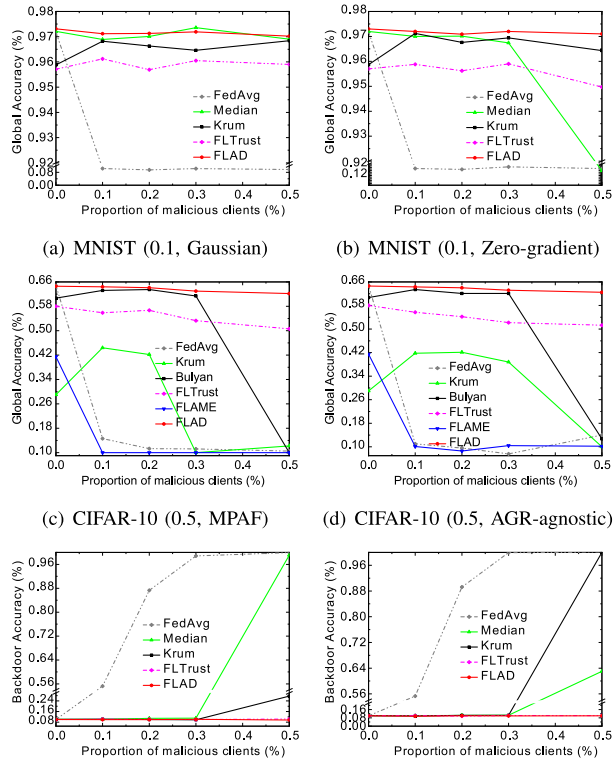
DATASET	FEDAVG	BULYAN	KRUM	FLAME	FLD	FLTRUST	FLAD	LOCAL TRAINING/ROUND
MNIST (0.8, 20%)	0.01	0.56	0.57	0.66	0.94	0.40/0.08	0.69/0.04	0.25
FEMNIST (20%)	0.02	0.77	0.81	0.87	1.06	0.58/0.12	0.74/0.04	0.36
CIFAR-10 (0.5, 20%)	0.06	12.54	11.53	1980.77	10.9	1.09/79.02	1.65/0.07	1.90

For FLTrust and FLAD, times are given as "server training/aggregation." the last column indicates client local training time per round, which can run parallel to server training.



(a) FLAD-NoDBSCAN (b) FLAD

Fig. 8. Comparison of feature vector classified by FLAD-NoDBSCAN and FLAD (with DBSCAN).



(a) MNIST (0.1, Gaussian) (b) MNIST (0.1, Zero-gradient)

(c) CIFAR-10 (0.5, MPAF) (d) CIFAR-10 (0.5, AGR-agnostic)

(e) MNIST (0.1, Backdoor-data) (f) MNIST (0.1, DBA)

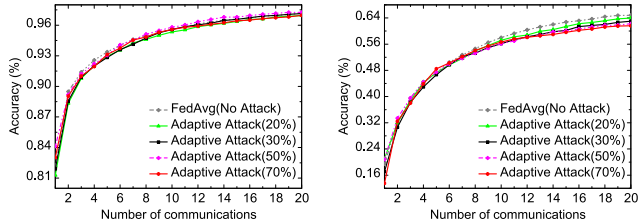
Fig. 9. Global and backdoor accuracy of different defences against different attacks across varying proportions of malicious clients.

shown in Fig. 8. Therefore, malicious detection effect of FLAD is better than FLAD-NoDBSCAN. In FLAD-NoFCNN, since local gradients belong to high-dimensional vectors, the direct use of DBSCAN to classify high-dimensional local gradients without using FEM dimensionality reduction will bring huge errors. In general, FLAD outperforms four ablation variants. Both DBSCAN and neural network Feature Extraction Models play an integral role in the performance improvement of FLAD.

4) Proportion of Malicious Client: The global and backdoor accuracy of FLAD hardly changes as the proportion of attackers increases, similar to FedAvg without attacks in Fig. 9. Although the global accuracy of FLTrust remains almost stable as the proportion of attackers increases, it is significantly lower than that of FLAD. The global accuracy of Median and Bulyan drops sharply when the proportion reaches 50%. Therefore, FLAD can tolerate up to 50% of malicious clients and has a large advantage over other defences.

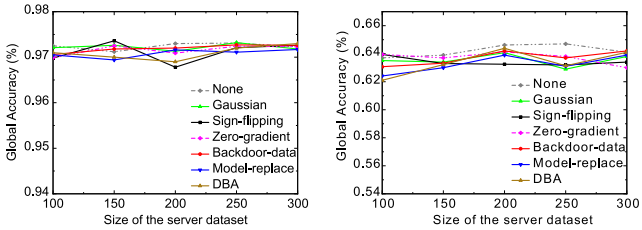
5) Comparison of Aggregation Time: As shown in Table VII, FLAD demonstrates significantly lower aggregation time compared to Bulyan, Krum, FLAME and FLD, showcasing a clear advantage in computational efficiency. While FLAD incurs slightly higher server model training time than FLTrust (e.g., 0.69 s vs. 0.40 s for MNIST and 1.65 s vs. 1.09 s for CIFAR-10), it exhibits a substantial advantage in aggregation time. Notably, on the complex CIFAR-10, FLAD maintains a low detection and aggregation time 0.07 s, which is only a fraction of FLTrust's 79.02 s. Moreover, server model training is independent of the clients and can be executed in parallel with local client training. For more complex datasets, where client local training time exceeds server model training time, the actual time consumption of FLAD and FLTrust depends primarily on their aggregation time. FLAD's aggregation time is significantly lower than FLTrust's. Overall, FLAD achieves efficient aggregation while greatly reducing the computational overhead for detection and aggregation on the server.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.



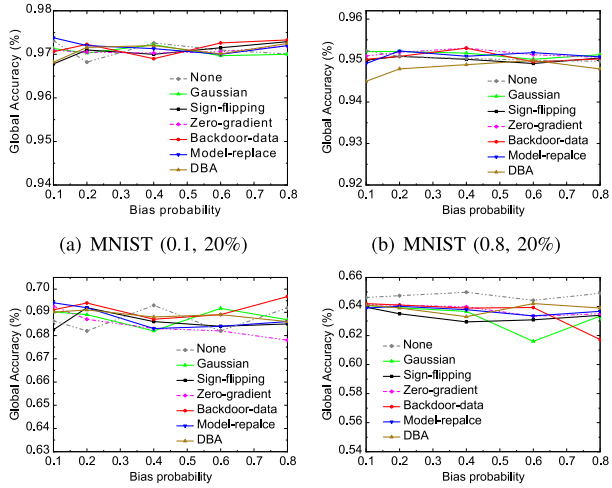
(a) MNIST (0.1) (b) CIFAR-10 (0.5)

Fig. 10. Global accuracy under adaptive attacks with different proportion of malicious clients.



(a) MNIST (0.1, 20%) (b) CIFAR-10 (0.5, 20%)

Fig. 11. The impact of server dataset size on global accuracy under different attacks.



(a) MNIST (0.1, 20%) (b) MNIST (0.8, 20%)

(c) CIFAR-10 (0.1, 20%) (d) CIFAR-10 (0.5, 20%)

Fig. 12. The impact of server dataset bias probability on global accuracy under different attacks.

6) Evaluation of Local Model Adaptive Attack: Adaptive attack on FLAD is described in Algorithm 5. In Fig. 10, even with malicious client proportions reaching 70%, FLAD's convergence rate and accuracy under adaptive attack resemble those of FedAvg without attacks. Therefore, the current state-of-the-art local model adaptive attack [14] proves ineffective against FLAD. When the attack strength is weak enough for FLAD to misjudge the malicious gradients, the malicious gradients are close enough to honest gradients to have almost no effect on the accuracy of aggregated global model.

7) Impact of the Server Dataset: We evaluate the impact of size and bias probability P of D_s on FLAD's global accuracy in Figs. 11 and 12. Fig. 11 shows the global accuracy of FLAD under different attacks as the D_s increases from 100 to 300, where D_s is sampled uniformly at random using Case I in Section V-A1. We observe that the global accuracy barely changes as the size of D_s changes, fluctuating slightly around 97.1% and 63.4% similar to FedAvg without attacks. This may be due to the fact that FLAD can successfully divide the malicious and honest clients using a small number of server dataset. Fig. 12 shows that for common attacks, changes of bias probability P of D_s do not affect the global accuracy, which is similar to FedAvg without attacks.

According to the experimental part of FLTrust, observing the global accuracy of FLTrust with different bias probabilities P, we find that FLTrust is more sensitive to changes in bias probabilities compared to FLAD. For MNIST (0.1), when P reaches 0.8, the test error rate of FLTrust under targeted attack is as high as 78% (global accuracy 22%) while FLAD still maintains 97%. This may be because FLAD learns honest features obtained from clean data, whereas FLTrust directly uses clean data to bootstrap trust. Therefore, compared with FLTrust, which also uses server dataset to bootstrap trust, our FLAD is more robust to the distribution transformations of server dataset.

VI. PFLAD: Privacy-Preserving FLAD

A. Privacy-Preserving FL

Although FL prevents clients from sharing their private data directly, the process of exchanging parameters can also leak the client's private information to attackers, resulting in *inference attack* [3], such as DLG [42], IDLG (improved deep leakage from gradients) [43] and GRNN (generative regression neural network) [44]. These inference attacks highlight the need for protection of parameters such as transmission gradients in FL.

Most existing privacy-preserving FL is based on well-known privacy-preserving techniques, including Homomorphic Encryption (HE), Secure Multi-Party Computation (SMPC), and Differential Privacy (DP) [3], [45]. PEFL [5] is a privacy-enhancing FL that uses Paillier encryption, but it requires to encrypt and decrypt up to four times between the server and the cloud platform in each iteration. BREA [46] uses the Feldman verifiable secret sharing, but it introduces significant multi-party communication and computational overhead. DP-FedAvg [47] uses DP with adaptive clipping, which protects privacy but introduces the accuracy penalty, while not being resistant to poisoning attacks. Moreover, there are privacy-preserving FL such as FedCG [48] and PPCL [49] based on the FL training itself. However, few schemes can achieve both Byzantine-robust against model poisoning attacks and privacy-preserving against inference attacks for FL.

B. The Framework of PFLAD

We use CKKS homomorphic encryption [9] which can handle floating-point numbers for transmission gradient privacy protection, thus achieving privacy-preserving FL. Moreover, in order to maintain local gradients secret to semi-honest clients and server, a Trusted Third Party (TTP) needs to be introduced for homomorphic encryption and decryption.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

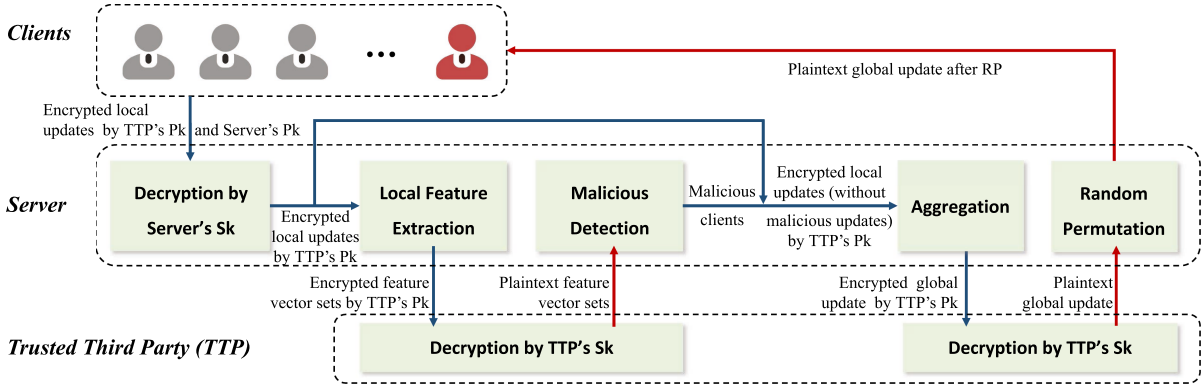


Fig. 13. The framework of PFLAD.

Fig. 13 shows the framework of PFLAD on the basis of FLAD, where red arrows represent plaintext and blue arrows represent ciphertext. Specifically, Client i encrypts local gradients $P_{ks}(P_{kt}(w_i^t))$ using the public key of TTP P_{kt} and server P_{ks}. After decryption by S_{ks}, the server uses homomorphic technology to input $P_{kt}(w_i^t)$ into FEM to obtain $P_{kt}(Pro_i^t)$, which is then sent to TTP for decryption $S_{kt}(P_{kt}(Pro_i^t)) = Pro_i^t$. TTP returns Pro_i^t to server and Algorithm 3 is executed to get malicious clients. After rejecting malicious updates, the server obtains aggregation global gradients $P_{kt}(W^t) = \text{FedAvg}(P_{kt}(w_i^t))$, $i \in [1, n-c]$. $P_{kt}(W^t)$ is sent to TTP for decryption, and then TTP uses P_{ks} to encrypt $P_{ks}(W^t)$ preventing direct leakage of the original global gradients W^t to clients. If semi-honest clients collude with each other, they may infer honest gradients from W^t, thus causing indirect information leakage for honest clients. Therefore, the server uses Random Permutation (RP) [49] to scramble global model W^t to mitigate privacy leakage caused by semi-honest clients. Finally, the server returns the global model after RP W_{RP}^t to clients.

Random Permutation (RP): Drawing on the PPCL [49], we use the RP of model parameters, which takes advantage of the computational characteristics of neural networks to achieve global model parameter transformation without affecting accuracy. RP is mainly applicable to scenarios with global models consisting of convolutional neural networks (CNN) and fully connected neural networks (FCNN). Specifically, let $f_i \in \mathbb{R}^{n_i \times n_{i+1}}$ denote the convolution kernel with input channel n_i and output channel n_{i+1}, and consider f_i as a two-dimensional matrix with $n_i \times n_{i+1}$. After randomizing the column index n_{i+1} of f_i, the row index n_{i+1} of $f_{i+1} \in \mathbb{R}^{n_{i+1} \times n_{i+2}}$ needs to be permuted in the same order. f_{i+1} can continue to randomize the column index n_{i+2}, then f_{i+2} row indexes will be permuted accordingly. Moreover, the RP can also be used for FCNN, where the weights and bias of the network are disrupted in the same order. Since the order of the global model with RP at each interaction is random, the semi-honest or malicious clients cannot obtain more sensitive information from global parameters W^t.

Remark: FEM and feature extraction values are in plaintext in PFLAD. However, recovering original d-dimensional gradients from one-dimensional feature extraction value is relative to solving a d-element primary equation, which is impossible in theory.

C. Security Analysis of PFLAD

The privacy and semantic security of the CKKS homomorphic encryption against chosen-plaintext attacks (IND-CPA security) guarantee the strong security of PFLAD [9]. In this section, we provide a simulator-based hybrid proof to further demonstrate that during the execution of PFLAD, the joint view of the server and TTP will not reveal any user's private data, except for the privacy information inferred from the computed results. Consistent with the threat model and related assumptions in previous literature [5], [49], we make the following assumptions.

Assumption 3: According to the threat model and the definition of the TTP, we assume that both the server and TTP are honest and curious, but the TTP will not engage in collusion attacks with the server or any client.

Assumption 4: We assume the presence of malicious attackers among the clients, while all clients are assumed to be honest and curious.

Proposition 2 ((Honest but Curious Server and TTP)): Given a client set $U = \{U_1, \ldots, U_n\}$ and a set $C = \{S, TTP\}$, where S represents the server. Define $\mathbf{REAL}_{\mathbf{C}}^{\mathbf{U}}$ as a random variable representing the joint view of the server and TTP in C during the actual execution of PFLAD. There exists a Probabilistic Polynomial Time (PPT) simulator $\mathbf{SIM}$ such that the output of $\mathbf{SIM}$ is computationally indistinguishable from $\mathbf{REAL}_{\mathbf{C}}^{\mathbf{U}}$:

$$\mathbf{REAL}_{\mathbf{C}}^{\mathbf{U}} \equiv \mathbf{SIM}, \tag{18}$$

where $\equiv$ denotes computational indistinguishable.

Proof: $\mathbf{REAL}_{\mathbf{C}}^{\mathbf{U}}$ encompasses all internal states and messages received by C during the execution of PFLAD. We employ the standard hybrid proof method similar to [50] to prove this proposition. We perform a series of subsequent modifications and operations on the random variables within $\mathbf{REAL}_{\mathbf{C}}^{\mathbf{U}}$ and define a PPT simulator $\mathbf{SIM}$ such that the output of $\mathbf{SIM}$ is computationally indistinguishable from $\mathbf{REAL}_{\mathbf{C}}^{\mathbf{U},\kappa}$. The detailed proof is as follows.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

Hybrid 1: Initialize a random variable to ensure that it is statistically indistinguishable from $\mathbf{REAL}_{\mathbf{C}}^{\mathbf{U}}$ in distribution.

Hybrid 2: In this hybrid, we simulate the change in the behavior of client $U_i \in U$ such that each client U_i encrypts a randomly selected vector $\boldsymbol{\alpha_i}$ using the TTP's public key P_{kt} instead of the original $\boldsymbol{w_i}$ (for honest clients) or $\boldsymbol{X_i}$ (for malicious clients). Since only the content of the ciphertext is altered, the IND-CPA security property of HE and the non-collusion setting between the server and TTP guarantee that this hybrid is indistinguishable from the previous one.

Hybrid 3: Building upon Hybrid 2, in this hybrid, we encrypt the encrypted random vectors $P_{kt}(\boldsymbol{\alpha_i})$ using the server's public key P_{ks}, rather than the original $P_{kt}(\boldsymbol{w_i})$ or $P_{kt}(\boldsymbol{X_i})$. Since only the content of the ciphertext is altered, this hybrid is indistinguishable from the previous one.

Hybrid 4: In this hybrid, we simulate the server's execution of local feature extraction algorithm. We modify the input to the algorithm, using $P_{kt}(\boldsymbol{\alpha})$ in place of $P_{kt}(\boldsymbol{w})$, where $\boldsymbol{\alpha} = \{\boldsymbol{\alpha_1}, \ldots, \boldsymbol{\alpha_n}\}$ and $\boldsymbol{w} = \{\boldsymbol{w_1}, \ldots, \boldsymbol{w_n}\}$. Since only the content of the ciphertext is altered, when performing homomorphic computation $\boldsymbol{F_{fc}}(P_{kt}(\boldsymbol{\alpha}))$, the IND-CPA security property of HE and the non-collusion setting between the server and TTP guarantee that this hybrid is indistinguishable from the previous one.

Hybrid 5: In this hybrid, we simulate the server's local gradient aggregation. For honest clients $U_i \in U$ where $i \in [1, n-c]$, we perform homomorphic addition and multiplication using randomly encrypted vectors $P_{kt}(\boldsymbol{\alpha_i})$ to obtain the aggregated value, instead of using $P_{kt}(\boldsymbol{w_i})$ for computation. Since only the content of the ciphertext is altered, when performing homomorphic computation $\frac{1}{n-c}\sum_{i=1}^{n-c}[P_{kt}(\boldsymbol{\alpha_i})]$, the IND-CPA security property of HE and the non-collusion setting between the server and TTP guarantee that this hybrid is indistinguishable from the previous one.

The proof proves that there exists a simulator $\mathbf{SIM}$ as described above so that its output is computationally indistinguishable from the output of $\mathbf{REAL}$. Therefore, PFLAD maintains privacy security for the server and TTP, meaning that honest but curious server and TTP cannot gain access to the private gradient information of clients.

Proposition 3 ((Clients).): PFLAD provides privacy security for clients, meaning that malicious or honest but curious clients cannot compromise the privacy of other clients.

Proof: Malicious or honest but curious clients attempt to infer the presence of target samples in the training dataset by exploiting incorrect gradient vectors [51]. This attack heavily relies on aggregation rules that involve only averages. In PFLAD, the globally aggregated gradients are sent to each client after being randomly permuted, shuffling the specific data arrangement within the gradients. As a result, malicious or honest but curious clients cannot infer whether target samples are used in the training. Therefore, PFLAD ensures privacy security for clients.

In conclusion, PFLAD enables the protection of FL transmission gradients and provides privacy security for all clients, server and TTP.

TABLE VIII
GLOBAL MODEL ACCURACY OF PFLAD UNDER DIFFERENT ATTACKS IN MNIST(0.1) AND MNIST(0.8)

	NONE	GAUSSIAN	SIGN-FLIPPING	ZERO-GRADIENT	MODEL-REPLACE
MNIST (0.1, 20%)	97.3	97.1	97.0	97.1	97.2
MNIST (0.1, 50%)	97.3	97.0	97.0	97.1	96.9
MNIST (0.8, 20%)	95.1	95.1	95.0	95.1	95.0

D. Performance Analysis of PFLAD

Experiments[4] on MNIST show that PFLAD achieves global model accuracy under different attacks similar to FLAD in Table VIII.

VII. DISCUSSION AND LIMITATIONS

Feature Extraction Model Training: The feature extraction model has to be trained in every iteration. The global model converges differently in each iteration, leading to varying honest gradient features. Therefore, it is necessary to train the feature extraction model for honest gradients in every iteration, which can be achieved for servers with abundant computational resources. However, we acknowledge that it is less friendly to computing-constrained servers.

The Quality of Server Dataset D_s: We acknowledge that the performance of FLAD suffers when D_s is poisoned. However, our FLAD only needs a small amount of clean D_S, easily obtainable by the server through methods like manual data generation and labeling [6].

Privacy in Feature Extraction Values: In our PFLAD, recovering the original d-dimensional local gradient vector from the one-dimensional feature extraction value is relative to solving a d-element primary equation. It is impossible to solve this equation in theory. However, it is an interesting future work that clients potentially leverage external knowledge to solve the equation, thus leading to a privacy breach of the original gradients.

VIII. CONCLUSION AND FUTURE WORK

We proposed FLAD, a Byzantine-robust FL defence based on gradient feature anomaly detection. Specifically, the server collects a small clean server dataset to bootstrap trust and trains Feature Extraction Models (FEM) to extract client local gradients features. DBSCAN is used to evaluate the similarity between the feature extraction values of clients. Through cosine similarity and relative length difference with standard features, malicious clients can be identified. Extensive experiments on Non-IID and IID datasets show FLAD achieves higher accuracy, robustness, and efficiency compared to state-of-the-art defences. Moreover, we implement privacy-preserving FLAD (PFLAD) using CKKS and Random Permutation to ensure transmitted gradient privacy. In future work, we will focus on improving the extraction accuracy and efficiency of FEM, while minimizing the requirements on server dataset, and further exploring ways to optimize the efficiency of our FLAD.

[4]**CKKS Settings:** Polynomial mod degree 8192. Modal size of the coefficients of the polynomial [40, 21, 21, 21, 21, 21, 21, 21, 40]. Global scaling factor 2^{21}. Galois Keys are introduced for batch rotation.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] P. Kairouz et al., "Advances and open problems in federated learning," 2019, *arXiv:912.04977*.

[2] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 2, pp. 1–19, 2019.

[3] L. Lyu et al., "Privacy and robustness in federated learning: Attacks and defenses," 2020, *arXiv: 2012.06337*.

[4] W. Jiang, H. Li, S. Liu, X. Luo, and R. Lu, "Poisoning and evasion attacks against deep learning algorithms in autonomous vehicles," *IEEE Trans. Veh. Technol.*, vol. 69, no. 4, pp. 4439–4449, Apr. 2020.

[5] X. Liu, H. Li, G. Xu, Z. Chen, X. Huang, and R. Lu, "Privacy-enhanced federated learning against poisoning adversaries," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4574–4588, 2021.

[6] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. System Secur. Symp.*, 2021.

[7] C. Xie, M. Chen, P.-Y. Chen, and B. Li, "CRFL: Certifiably robust federated learning against backdoor attacks," in *Proc. Int. Conf. Mach. Learn.*., 2021, pp. 11372–11382.

[8] T. D. Nguyen et al., "FLAME: Taming backdoors in federated learning," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 1415–1432.

[9] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Proc. 23rd Int. Conf. Theory Appl. Cryptol. Inf. Secur.*, Hong Kong, China, 2017, pp. 409–437.

[10] F. Lin, Q. Ling, and Z. Xiong, "Byzantine-resilient distributed large-scale matrix completion," in *Proc. IEEE Int. Conf. Acoust. Speech Signal Process.*, 2019, pp. 8167–8171.

[11] Z. Wu, Q. Ling, T. Chen, and G. B. Giannakis, "Federated variance-reduced stochastic gradient descent with robustness to byzantine attacks," *IEEE Trans. Signal Process.*, vol. 68, pp. 4583–4596, 2020.

[12] L. Li, W. Xu, T. Chen, G. B. Giannakis, and Q. Ling, "RSA: Byzantine-robust stochastic aggregation methods for distributed learning from heterogeneous datasets," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 1544–1551.

[13] X. Cao and N. Z. Gong, "MPAF: Model poisoning attacks to federated learning based on fake clients," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 3396–3404.

[14] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to byzantine-robust federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1605–1622.

[15] V. Shejwalkar and A. Houmansadr, "Manipulating the byzantine: Optimizing model poisoning attacks and defenses for federated learning," in *Proc. Netw. Distrib. System Secur. Symp.*, 2021. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/manipulating-the-byzantineoptimizing-model-poisoning-attacks-and-defenses-for-federated-learning/

[16] B. Biggio, B. Nelson, and P. Laskov, "Poisoning attacks against support vector machines," 2012, *arXiv:1206.6389*.

[17] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," in *Proc. Eur. Symp. Res. Comput. Secur.*, 2020, pp. 480–501.

[18] E. Bagdasaryan, A. Veit, Y. Hua, D. Estrin, and V. Shmatikov, "How to backdoor federated learning," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2020, pp. 2938–2948.

[19] Z. Sun, P. Kairouz, A. T. Suresh, and H. B. McMahan, "Can you really backdoor federated learning?," 2019, *arXiv: 1911.07963*.

[20] C. Xie, K. Huang, P.-Y. Chen, and B. Li, "DBA: Distributed backdoor attacks against federated learning," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=rkgyS0VFvr

[21] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Arti. Intelli. Statist.*, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[22] P. Blanchard, E. M. El Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 118–128.

[23] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5650–5659.

[24] R. Guerraoui and S. Rouault, "The hidden vulnerability of distributed learning in byzantium," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 3521–3530.

[25] S. Rajput, H. Wang, Z. Charles, and D. Papailiopoulos, "DETOX: A redundancy-based framework for faster and more robust gradient aggregation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 10320–10330.

[26] K. Pillutla, S. M. Kakade, and Z. Harchaoui, "Robust aggregation for federated learning," *IEEE Trans. Signal Process.*, vol. 70, pp. 1142–1154, 2022.

[27] A. N. Bhagoji, S. Chakraborty, P. Mittal, and S. Calo, "Analyzing federated learning through an adversarial lens," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 634–643.

[28] J. Bernstein, J. Zhao, K. Azizzadenesheli, and A. Anandkumar, "signSGD with majority vote is communication efficient and fault tolerant," 2018, *arXiv: 1810.05291*.

[29] C. P. Wan and Q. Chen, "Robust federated learning with attack-adaptive aggregation," 2021, *arXiv:2102.05257*.

[30] M. Barreno, B. Nelson, A. D. Joseph, and J. D. Tygar, "The security of machine learning," *Mach. Learn.*, vol. 81, no. 2, pp. 121–148, 2010.

[31] M. Jagielski, A. Oprea, B. Biggio, C. Liu, C. Nita-Rotaru, and B. Li, "Manipulating machine learning: Poisoning attacks and countermeasures for regression learning," in *Proc. 2018 IEEE Symp. Secur. Privacy*, 2018, pp. 19–35.

[32] Z. Zhang, X. Cao, J. Jia, and N. Z. Gong, "FLDetector: Defending federated learning against model poisoning attacks via detecting malicious clients," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 2545–2555.

[33] S. Shen, S. Tople, and P. Saxena, "AUROR: Defending against poisoning attacks in collaborative deep learning systems," in *Proc. 32nd Annu. Conf. Comput. Secur. Appl.*, 2016, pp. 508–519.

[34] R. Ganjoo, M. Ganjoo, and M. Patil, *Mitigating Poisoning Attacks in Federated Learning*. Berlin, Germany: Springer, 2022, pp. 687–699.

[35] P. Rieger, T. D. Nguyen, M. Miettinen, and A.-R. Sadeghi, "DeepSight: Mitigating backdoor attacks in federated learning through deep model inspection," in *Proc. Netw. Distrib. System Secur. Symp.*, 2022. [Online]. Available: https://www.ndss-symposium.org/ndss-paper/auto-draft-205/

[36] E. Schubert, J. Sander, M. Ester, H. P. Kriegel, and X. Xu, "DBSCAN revisited, revisited: Why and how you should (still) use dbscan," *ACM Trans. Database Syst.*, vol. 42, no. 3, pp. 1–21, 2017.

[37] M. D. Zeiler, "Adadelta: An adaptive learning rate method," 2012, *arXiv:1212.5701*.

[38] Y. LeCun, C. Cortes, and C. J. Burges, "Mnist handwritten digit database," 1998. [Online]. Available: http://yann.lecun.com/exdb/mnist

[39] A. Krizhevsky and G. Hinton, "Learning multiple layers of features from tiny images," Univ. Toronto, Toronto, Ontario, Tech. Rep., 2009. [Online]. Available: https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf

[40] S. Caldas et al., "LEAF: A benchmark for federated settings," 2018, *arXiv: 1812.01097*.

[41] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," 2014, *arXiv:1412.6572*.

[42] L. Zhu, Z. Liu, and S. Han, "Deep leakage from gradients," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 14774–14784.

[43] B. Zhao, K. R. Mopuri, and H. Bilen, "IDLG: Improved deep leakage from gradients," 2020, *arXiv: 2001.02610*.

[44] H. Ren, J. Deng, and X. Xie, "GRNN: generative regression neural network — a data leakage attack for federated learning," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 4, pp. 1–24, 2022.

[45] H. Fereidooni et al., "SAFELearn: Secure aggregation for private federated learning," in *Proc. 2021 IEEE Secur. Privacy Workshops*, 2021, pp. 56–62.

[46] J. So, B. Güler, and A. S. Avestimehr, "Byzantine-resilient secure federated learning," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 7, pp. 2168–2181, Jul. 2021.

[47] G. Andrew, O. Thakkar, B. McMahan, and S. Ramaswamy, "Differentially private learning with adaptive clipping, in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 17455–17466.

[48] Y. Wu et al., "FedCG: Leverage conditional GAN for protecting privacy and maintaining competitive performance in federated learning," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 2334–2340.

[49] H. Yan, L. Hu, X. Xiang, Z. Liu, and X. Yuan, "PPCL: Privacy-preserving collaborative learning for mitigating indirect information leakage," *Inf. Sci.*, vol. 548, pp. 423–437, 2021.

[50] G. Xu, H. Li, Y. Zhang, S. Xu, J. Ning, and R. H. Deng, "Privacy-preserving federated deep learning with irregular users," *IEEE Trans. Dependable Secure Comput.*, vol. 19, no. 2, pp. 1364–1381, Mar./Apr. 2022.

[51] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *Proc. 2019 IEEE Symp. Secur. Privacy*, 2019, pp. 739–753.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.

Peng Tang received the MS degree in computer science from the Beijing University of Posts and Telecommunications in 2017 and the PhD degree in cyber security from Shanghai Jiao Tong University in 2022. He is a postdoctoral with the School of Cyber Science and Engineering, Shanghai Jiao Tong University. His research interests include focuses on privacy protection, data science, and AI security.

Xiaoyu Zhu received the BS degree in information security from Xidian University, Xi'an, China, in 2022. She is currently working toward the master's degree with the School of Cyber Science and Engineering, Shanghai Jiao Tong University, Shanghai, China. Her research interests include focuses on privacy protection, data science, and AI security.

Weidong Qiu (Member, IEEE) received the MS degree in cryptography from Xidian University, Xi'an, China, in 1998, and the PhD degree in computer software theory from Shanghai Jiao Tong University, Shanghai, China, in 2001. He is currently a professor and doctoral supervisor with the School of Cyber Science and Engineering, Shanghai Jiao Tong University. His main research interests include data science, privacy computing, cryptography, and computer forensics.

Zheng Huang received the PhD degree in computer science from Shanghai Jiao Tong University, Shanghai, China, in 2003. He is currently an associate professor with the School of Cyber security, Shanghai Jiao Tong University. His main research interests include machine learning and security.

Zhenyu Mu received the BS degree in information security from Shanghai Jiao Tong University, Shanghai, China, in 2022. He is currently working toward the master's degree with the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research focuses on graph neural networks and data mining.

Shujun Li (Senior Member, IEEE) is professor in cyber security with the School of Computing and director of the Institute of Cyber Security for Society (iCSS), University of Kent, U.K. He has published more than 100 scientific papers, including five Best Papers. His research interests include about interdisciplinary topics related to cyber security and privacy, human factors, digital forensics and cybercrime, multimedia computing, AI and data science, and more recently education and learning. His work covers multiple application domains, including but not limited to digital health, smart cities, smart homes, and e-tourism. He is a fellow of the BCS, and a co-founder and a vice president with the Association of British Chinese Professors (ABCP). He received multiple awards and honours including the 2022 *IEEE Transactions on Circuits and Systems Guillemin*-Cauer Best Paper Award and a 2022 Research and Innovation Award from the IEEE SMC Society TCHS.

Authorized licensed use limited to: HANGZHOU DIANZI UNIVERSITY. Downloaded on March 24,2026 at 13:09:07 UTC from IEEE Xplore. Restrictions apply.